# Alternative Reprogramming for Service Models

Yunbei Zhang[1] Chengyi Cai[2] Feng Liu[2] Jihun Hamm[1]
[1]Tulane University [2]University of Melbourne

## Abstract

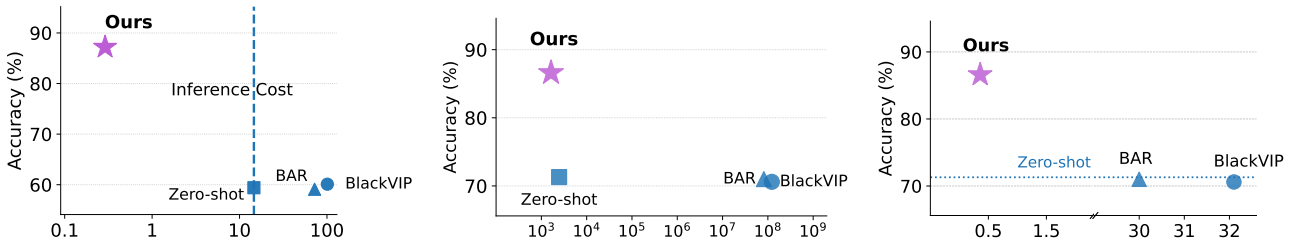
*Adapting closed-box service models (i.e., APIs) for target tasks typically relies on reprogramming via Zeroth-Order Optimization (ZOO). However, this standard strategy is known for extensive, costly API calls and often suffers from slow, unstable optimization. Furthermore, we observe that this paradigm faces new challenges with modern APIs (e.g., GPT-4o). These models can be less sensitive to the input perturbations ZOO relies on, thereby hindering performance gains. To address these limitations, we propose an Alternative efficient Reprogramming approach for Service models (AReS). Instead of direct, continuous closed-box optimization, AReS initiates a single-pass interaction with the service API to prime an amenable local pre-trained encoder. This priming stage trains only a lightweight layer on top of the local encoder, making it highly receptive to the subsequent glass-box (white-box) reprogramming stage performed directly on the local model. Consequently, all subsequent adaptation and inference rely solely on this local proxy, eliminating all further API costs. Experiments demonstrate AReS's effectiveness where prior ZOO-based methods struggle: on GPT-4o, AReS achieves a +27.8% gain over the zero-shot baseline, a task where ZOO-based methods provide little to no improvement. Broadly, across ten diverse datasets, AReS outperforms state-of-the-art methods (+2.5% for VLMs, +15.6% for standard VMs) while reducing API calls by over 99.99%. AReS thus provides a robust and practical solution for adapting modern closed-box models.* Code will be released.

## 1. Introduction

In recent years, Model-as-a-Service (MaaS) has emerged as a dominant paradigm for deploying and accessing state-of-the-art (SOTA) machine learning models [1, 5, 39, 46]. Companies and research institutions often release their large pre-trained models (which we refer to as *service models* in this work) as inference APIs, allowing users to query these powerful models without direct access to model parameters or architecture details. This closed-box nature creates a significant challenge for transfer learning [44, 45]. Unlike some approaches that may assume access to intermediate features or token embeddings [34, 50] (see Table 1 for a detailed comparison), this work focuses on the most restrictive closed-box setting, where only raw input-prediction access to the service model is available. In this challenging scenario, adapting models to downstream tasks, which typically requires gradient information, becomes particularly difficult. Model Reprogramming (MR) [7, 10, 22, 35, 47], known as Visual Reprogramming (VR) or Visual Prompting (VP) in vision domains, has been developed as one solution, enabling adaptation of MaaS models through input-level modifications optimized using zeroth-order optimization (ZOO) [28, 41, 42, 49] based solely on model outputs.

While closed-box visual reprogramming approaches such as BAR [47] and BlackVIP [33] have demonstrated promising results under this strict input-prediction only setting, they entail practical challenges. The optimization process, reliant on approximate gradients, can be slow and unstable, and the computational and practical costs could be prohibitive [29, 57]. Such methods require numerous API calls not only during the lengthy training phase but also for every inference instance, leading to significant financial burdens and requiring uninterrupted connectivity to the service model [33, 47, 50]. More critically, our experiments indicate that this ZOO-based paradigm faces new challenges with modern, robust APIs. We find that powerful models like GPT-4o can be less sensitive to the noisy input perturbations central to ZOO, yielding little to no performance gains (Fig. 1a and Table 4), while on models like LLaVA [27], these perturbations may affect delicate vision-language alignment. Furthermore, the substantial resources expended for these SOTA methods do not always translate into commensurate performance enhancements (Fig. 1b and 1c), raising concerns about their efficiency-to-effectiveness ratio [33, 53]. These substantial hurdles in efficiency, cost, and now, effectiveness on modern models, form the core motivation for our work. It compels us to ask: *Are there alternative approaches that can offer advantages beyond conventional closed-box model reprogramming, particularly in overcoming these practical and economic limitations?*

We propose an **A**lternative efficient **Re**programming approach for **S**ervice models (**AReS**) that directly addresses these challenges (see Fig. 2). Instead of direct, continuous




(a) **GPT-4o Cost (USD)** (b) **#API Calls** (c) **Training Time (hour)**

Figure 1. **(a)** On the real-world GPT-4o API, ZOO-based methods show limited effectiveness, providing little to no improvement over zero-shot performance while incurring high total (training and inference) costs. **(b, c)** On CLIP ViT-B/16, these methods require $\sim 10^8$ API calls and over 32 hours of training on Flowers102, yet still underperform ours, AReS, which uses only $\sim 10^3$ API calls and 0.4 hours.

closed-box optimization, AReS initiates a single-pass interaction with the service API to prepare an amenable local pre-trained encoder (e.g., a publicly available one, making the same initial assumption as methods like BlackVIP [33] but utilizing it for a fundamentally different purpose). This priming stage trains only a lightweight layer on top of the local encoder. Crucially, this step does not require a shared label space; it successfully prepares the local model for reprogramming even when the service API's outputs are in a disjoint label space (e.g., ImageNet) from the target task (e.g., Flowers). This approach, supported by a theoretical analysis connecting priming faithfulness to downstream performance, enables effective glass-box visual reprogramming to occur entirely and efficiently on the local model. Consequently, all subsequent adaptation and inference rely solely on this local model, eliminating further API costs. For example, on the Flowers102 dataset (as detailed in Fig. 1b and 1c), our approach achieves 86.6% accuracy, outperforming BlackVIP (70.6%), while dramatically reducing API calls from $\sim 10^8$ down to $\sim 10^3$ (>**99.99**% reduction) and slashing computation time from over 30 hours to less than half an hour (>**98.88**% reduction). Crucially, on GPT-4o, AReS improves by $+27.8\%$, while BlackVIP gains only $+0.7\%$. This is because the service model's robustness to input perturbations hinders ZOO-based methods, a limitation AReS bypasses by avoiding direct reprogramming.

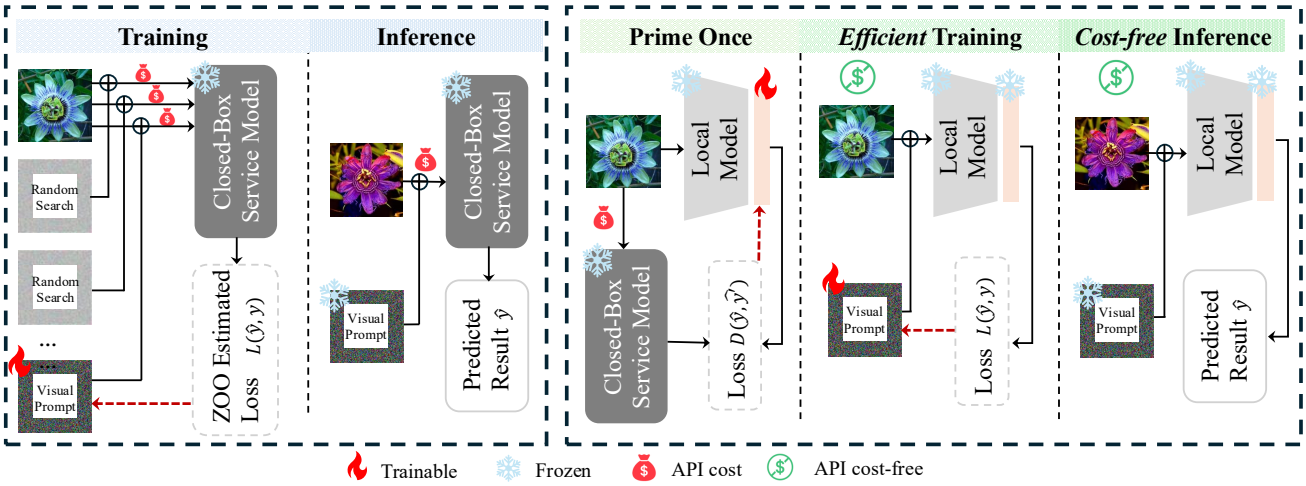
We evaluate AReS across a broad range of service models. Our primary evaluation spans standard Vision Models (VMs) (e.g., ViT-B/16 [12], ResNet101 [15]) and Vision-Language Models (VLMs) (e.g., CLIP [39]) over ten diverse datasets. We also conduct evaluations on modern models, including the Multimodal Large Language Models (MLLMs) LLaVA [27], the proprietary MLLM API GPT-4o [1], and the commercial VM API Clarifai. Through these comprehensive experiments, we demonstrate that our approach consistently outperforms SOTA ZOO-based methods while dramatically reducing computational requirements and associated costs. These findings have important implications for the broader field of transfer learning in increasingly API-centric machine learning ecosystems, suggesting a more effective path forward that balances performance, efficiency, and practicality. Our main contributions are summarized as follows:

1. We identify a key challenge for ZOO-based reprogramming on modern, robust APIs and present an effective alternative that shifts adaptation to a local model after a minimal, single-pass interaction with the service API.
2. We introduce **AReS**, a simple, theoretically-supported framework that combines a priming step with efficient glass-box reprogramming, uniquely enabling *cost-free* inference without further API dependency.
3. Through extensive experiments, AReS achieves superior performance, including significant gains on real-world APIs (e.g., GPT-4o and Clarifai) where prior methods fail, alongside average accuracy improvements of $+2.5\%$ for VLM and $+15.6\%$ for VM over BlackVIP, while reducing API calls by $> 99.99\%$.

## 2. Related Work

**Closed-box Tuning for Model-as-a-Service (MaaS).** Closed-box tuning for MaaS has been explored across various modalities, with approaches for Large Language Models (LLMs) [44, 45] and methods for VLMs that modify inputs at the token level while requiring access to text embeddings, placing them outside the strictest closed-box definition [14, 34, 50, 53], as detailed in Table 1. In the vision domain, approaches like BlackVIP [33] and BAR [47] employ ZOO to reprogram models by modifying visual inputs. While applicable to both VLMs and VMs, these ZOO-based approaches are known for optimization instability and high API call costs during training and inference [29, 33, 35, 47, 57]. More critically, their reliance on input perturbations may face new challenges in the era of modern APIs, e.g., powerful service models like GPT-4o can be robust enough to simply ignore these perturbations, rendering the costly ZOO adaptation process ineffective (Fig. 1a). These limitations highlight the need for an alternative strategy that avoids direct closed-box optimization and instead enables efficient and effective adaptation for downstream tasks without repeated service-model queries.

**Model Reprogramming (MR).** MR [3, 10, 22, 56], termed




**(a) Previous Closed-box VR** **(b) AReS (Ours)**

Figure 2. **(a)** Previous closed-box methods use Zeroth-Order Optimization (ZOO), which estimates gradients by querying the API multiple times for the same training image with random perturbations. This results in continuous API dependency: numerous calls during training and one API call per image during inference. **(b)** Our AReS approach performs a single-pass priming, requiring only one API call per training image to prepare a local model. This enables efficient, gradient-aware VR locally, eliminating all subsequent API costs during inference.

Visual Reprogramming (VR) or Visual Prompting (VP) in vision domains, encompasses two main strategies. The first, token-based prompting, is tailored for transformer architectures like Vision Transformers (ViT) [3, 22, 54, 58, 59]. These methods inject learnable tokens and require access to the model's embedding space or intermediate layers, rendering them unsuitable for strict closed-box settings or non-transformers [14, 34, 50, 53]. The second category, input-level prompting [6, 8, 33, 47, 48], directly modifies input images, offering model-agnostic flexibility with only input-output access. Both approaches can be optimized using gradient-based methods (glass-box) or ZOO techniques (closed-box) [33, 47, 50]. However, ZOO-based methods can be associated with efficiency and stability trade-offs. Their reliance on high query volumes and continuous API connectivity may be less suited for practical scenarios requiring rapid adaptation or offline use, such as in remote field operations. These practical considerations particularly motivate our work in exploring an alternative strategy.

## 3. Problem Setting

**Closed-box Model Reprogramming.** MR adapts pre-trained models to solve new tasks without modifying their parameters. Given a pre-trained model $\mathcal{F}_S : \mathcal{X}^S \mapsto \mathbb{R}^{K^S}$ where $K^S$ is the number of source classes, the key concept is to introduce a learnable prompt $\mathbf{P}$ that transforms inputs from the downstream task domain $\mathcal{X}^T$ to the source domain $\mathcal{X}^S$ through an input transformation function $g_{\text{in}}(\cdot|\mathbf{P})$. Additionally, a label mapping function $g_{\text{out}} : \mathbb{R}^{K^S} \mapsto \mathbb{R}^{K^T}$ is employed to bridge the source (with $K^S$ classes) and target (with $K^T$ classes) label spaces. If the source and downstream tasks have the same label space, $g_{\text{out}}$ becomes an identity mapping (e.g., in VLMs). Otherwise, for VMs, a gradient-free label mapping is established following existing works [6, 8, 47] (Detailed in Appendix 7.2). Importantly, only the input transformation $g_{\text{in}}$ contains trainable parameters updated through gradient descent, while the label mapping $g_{\text{out}}$ is computed and updated during training but does not introduce additional trainable parameters.

In MaaS settings, powerful pre-trained models are accessible only through APIs without access to their parameters or internal representations. We denote such a closed-box Service API model as $\mathcal{F}_S : \mathcal{X}^S \mapsto \mathbb{R}^{K^S}$, which accepts inputs from domain $\mathcal{X}^S$ and produces output predictions. Adhering to the strictest closed-box assumption established in prior work [47], we consider the scenario where the API returns only prediction probabilities rather than logits or embeddings, representing the most constrained access scenario (see Table 1). When adapting MaaS models to downstream tasks, the central challenge is to leverage these powerful models in a computationally and economically efficient manner. This requires developing methods that can optimize task-specific visual prompts using only the limited input-output responses from the API while minimizing computational costs and API usage during both training and inference.

**Existing BMR Approaches.** Existing ZOO-based BMR methods like BAR [47] employs random gradient-free (RGF) optimization [28, 49] to estimate the gradient of the loss function $\ell$ with respect to prompt parameters $\mathbf{P}$: $\nabla\ell(\mathbf{P}) \approx \frac{d}{q}\sum_{i=1}^{q}\frac{\ell(\mathbf{P}+\mu u_i)-\ell(\mathbf{P})}{\mu}u_i$, where $d$ is the parameter dimension, $q$ is the number of random directions, $\mu$ is a small constant, and $u_i$ are random unit vectors. BlackVIP [33] introduces the Simultaneous Perturbation

Table 1. Comparison of tuning methods for Model-as-a-Service (MaaS). The Access column indicates the strictness, distinguishing translucent-box (grey, requiring intermediate Features or Embedding access) from closed-box (black, Input/Predictions only) settings. We compare methods by: MR (Model Reprogramming-based); required Input Access and Output Access; support for VMs (Standard Vision Models) and VLMs (Vision-Language Models); Prompt type (L: Language, V: Visual); and Encoder Flexibility. Our method, AReS, uniquely offers Cost-free Inference for both VMs and VLMs while operating under the strictest closed-box constraints.

| Access | Method | MR | Input Access | Output Access | VM | VLM | Prompt | Encoder Flexibility | Cost-free Inference |
|---|---|---|---|---|---|---|---|---|---|
| | LFA [34] | ✗ | Input | Features | ✗ | ✓ | – | ✓ | ✗ |
| | CBBT [14] | ✗ | Embedding | Features | ✗ | ✓ | L | ✓ | ✗ |
| | LaViP [26] | ✓ | Input | Features | ✓ | ✓ | V | ✓ | ✗ |
| | BPT-VLM [53] | ✗ | Embedding | Predictions | ✗ | ✓ | L&V | ViT Only | ✗ |
| | CraFT [50] | ✗ | Embedding | Predictions | ✗ | ✓ | L | ✓ | ✗ |
| | BAR [47] | ✓ | Input | Predictions | ✓ | ✓ | V | ✓ | ✗ |
| | BlackVIP [33] | ✓ | Input | Predictions | ✓ | ✓ | V | ✓ | ✗ |
| | LLM-Opt [30] | ✗ | Input | Predictions | ✗ | ✓ | L | ✓ | ✗ |
| | **AReS (Ours)** | ✓ | Input | Predictions | ✓ | ✓ | V | ✓ | ✓ |

Stochastic Approximation with Gradient Correction (SPSA-GC) [41, 42] to improve gradient estimation: $\nabla\ell(\mathbf{P}) \approx \frac{\ell(\mathbf{P}+c\Delta)-\ell(\mathbf{P}-c\Delta)}{2c}\Delta^{-1}$ , where $c$ is a small constant and $\Delta$ is a random perturbation vector. BlackVIP also incorporates a Coordinator network for input-dependent prompt generation using a local pre-trained encoder (e.g., ViT-B/16). While these methods have shown promising results, they require multiple API calls per update and during inference, leading to increased computational and financial costs. Additionally, their zeroth-order gradient approximation can result in less stable optimization compared to exact gradients (i.e., first-order optimization) [29, 33, 57]. The challenge remains to develop more efficient approaches that can better leverage available local encoders while minimizing API usage.

## 4. Method

**Prime Once.** In the first stage, we utilize a locally available encoder $\mathcal{F}_L$ (the same assumption as BlackVIP [33]) and prime it using the closed-box service model $\mathcal{F}_S$. For each image $x_i$ from the target domain dataset $\mathcal{D}^T = \{x_i\}_{i=1}^N$, we query the service model to obtain prediction probabilities $p_S(x_i) = \mathcal{F}_S(x_i)$. It is important to note that this is the *only time* we interact with the API, and after this single round of querying, the API is *no longer needed* for the rest of the process. We use the local encoder $\mathcal{F}_L$ (outputting $d_{\text{enc}}$-dim features) as a fixed feature extractor and add a trainable linear layer with parameters $\theta \in \mathbb{R}^{K^S \times (d_{\text{enc}}+1)}$ on top of it. Note that we abuse the notation $\mathcal{F}_L$ to refer to both the local encoder alone and the local encoder combined with the learned linear layer, depending on the context. Only the linear layer parameters are updated during priming:

$$\theta^* = \arg\min_\theta \sum_{i=1}^N \mathcal{L}_{\text{P}}(\mathcal{F}_L(x_i;\theta), p_S(x_i)) \,. \tag{1}$$

Our priming objective, inspired by the loss function from knowledge distillation [17, 43], is defined as: $\mathcal{L}_{\text{P}}(p_L, p_S) := -\sum_{j=1}^{K^S} p_{S,j} \log p_{L,j}$ , where $p_S/p_L$ are the output probabilities of the same image from the service/local models, respectively. This loss is equivalent to the Kullback-Leibler divergence $\text{KL}(p_S||p_L)$.

For both VMs and VLMs, we freeze the local encoder and only train the added linear layer. The key difference between VM and VLM adaptation is in how the outputs are handled: for VMs, the priming occurs in the *service model label probability space*, which presents a non-trivial challenge as this fixed vocabulary does not align with the downstream task's labels. For VLMs, the outputs are naturally aligned with the downstream task label space through the text encoder. This approach establishes a fair comparison with BlackVIP [33], as both methods only require a local encoder. Moreover, this opens up the potential to utilize extra **unlabeled** data from the downstream distribution, a significant advantage unavailable to other methods. (see Fig. 3d).

**Remark** (Priming vs. Knowledge Distillation)**.** Our priming stage is mechanistically similar to knowledge distillation [2, 9, 17, 19, 21, 38] but differs fundamentally in its objective and applicability. Whereas traditional distillation aims to create a final, high-performance student model—typically requiring a shared label space—our priming is solely a preparatory step to make the local model more amenable to subsequent reprogramming. This preparatory focus allows priming to operate even when the service and downstream label spaces are disjoint (e.g., using Flowers data in the ImageNet label space), providing crucial flexibility for closed-box model adaptation.

**Efficient VR Training.** After the priming, we proceed to the second stage of AReS: learning an effective visual prompt for the downstream task using gradient-based optimization. Unlike closed-box approaches that rely on approximate gradients, we can leverage exact gradient information since our local encoder with the learned linear layer is fully accessible. We define a learnable visual prompt $\mathbf{P}$ and an input transformation function $g_{\text{in}}$ similar to closed-box methods, along with a label mapping function $g_{\text{out}}$ (and becomes an identity mapping for VLMs). We optimize the prompt using gradient descent on our primed local encoder by minimizing

Table 2. Accuracy and Efficiency comparison on ten diverse visual recognition datasets using CLIP ViT-B/16 (Service) and ViT-B/16 (Local) in a 16-shot setting. AReS achieves superior average accuracy with a drastic reduction in resources (99.99% fewer API calls and 98% less training time). For this and all subsequent tables, Gray denotes glass-box Service VR results. #API (M): total API calls in millions (training and inference); Wall-clock Time (h): total training hours run on an RTX 6000 Ada GPU.

| Method | Flowers | DTD | UCF | Food | GTSRB | EuroSAT | Pets | Cars | SUN | SVHN | Avg. | #API (M) | Time (h) |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| VR (glass-box) | 86.8 | 62.0 | 74.0 | 81.6 | 65.1 | 90.9 | 90.7 | 66.2 | 66.7 | 60.2 | 74.4 | 16.82 | 9.8 |
| Zero-shot | 71.3 | 43.9 | 66.9 | **85.9** | 21.0 | 47.9 | **89.1** | 65.2 | 62.6 | 17.9 | 57.2 | 0.12 | 0 |
| BAR | 71.0 | 46.8 | 64.2 | 84.4 | 21.5 | 77.3 | 88.4 | 63.0 | 62.4 | 34.6 | 61.4 | 612.84 | 185.6 |
| BlackVIP | 70.6 | 45.3 | **68.7** | **85.9** | 21.3 | 73.3 | **89.1** | **65.4** | **64.5** | 44.4 | 62.9 | 754.20 | 197.5 |
| LLM-Opt | 67.6 | 45.0 | 59.9 | 78.5 | 21.2 | 48.0 | 87.7 | 56.2 | 60.3 | 20.2 | 54.5 | 5011.32 | 5.1 |
| **AReS** | **86.6** | **48.2** | 67.1 | 68.8 | **39.4** | **85.7** | 88.9 | 43.2 | 62.8 | **63.2** | 65.4 | **0.02** | **3.7** |
| **AReS-MS** | **86.6** | **48.2** | 67.1 | **85.9** | **39.4** | **85.7** | 88.9 | 65.2 | 62.8 | **63.2** | **69.3** | **0.06** | **3.7** |

the cross-entropy loss:

$$\mathbf{P}^* = \arg\min_{\mathbf{P}} \mathbb{E}_{(x,y)\sim\mathcal{D}^T}[\ell(g_{\text{out}}(\mathcal{F}_L(g_{\text{in}}(x,\mathbf{P});\theta^*)),y)]\,. \tag{2}$$

This gradient-based optimization on local model enables more stable, efficient, and rapid learning of the visual prompt compared to ZOO-based methods, resulting in better and faster adaptation to the downstream task while maintaining the benefits of input-level modifications.

**Cost-free Inference.** For inference on test examples from the downstream task, we apply the learned visual prompt to the input and process it through the local encoder $\hat{y} = g_{\text{out}}(\mathcal{F}_L(g_{\text{in}}(x,\mathbf{P}^*);\theta^*))$. This approach eliminates API dependency during inference, enabling truly cost-free deployment. By removing the need for constant connectivity to service models, AReS becomes particularly valuable in practical scenarios like remote field operations or edge devices with intermittent network access.

**Model Selection Strategy.** For applications where maximizing performance is the top priority, we also propose a simple variant named **AReS-MS** (Model Selection). This approach leverages the initial priming stage as a low-cost diagnostic tool, as it yields both the zero-shot performance of the service API and the potential performance of the primed local model. AReS-MS then automatically selects the optimal inference path: if the local model's potential meets the required performance threshold, defined as the zero-shot performance minus a tolerance $\tau$, it proceeds with the cost-free local model; otherwise, it defaults to using the more powerful zero-shot API. This turns AReS into a more complete and robust framework that not only enables efficient adaptation but also guides practitioners to the optimal cost-performance trade-off for each individual task.

**Theoretical Insight.** To formally understand AReS's efficacy, we provide a theoretical analysis (detailed in Appendix 9) that bounds the performance of our primed local model, $\mathcal{F}_L$, relative to the service model, $\mathcal{F}_S$. This analysis hinges on two key assumptions: *Service Model Superiority* (Assumption 1), which reasonably posits that $\mathcal{F}_S$ is more powerful or better aligned with the downstream task data $\mathcal{D}^T$ than $\mathcal{F}_L$ initially, and $\epsilon$-*Faithful Priming* (Assumption 2 and Definition 1). The latter states our one-time priming effectively aligns the output logit distributions of $\mathcal{F}_L$ and $\mathcal{F}_S$, with their expected $L_1$ difference bounded by a small $\epsilon$, indicating a successful knowledge transfer. While our theoretical analysis assumes *logits* for tractability, our ablation studies (Fig. 3c) confirm that the performance of our practical, probability-only method is unaffected.

Leveraging these assumptions and the Lipschitz continuity of the cross-entropy loss (Lemma 1), Theorem 3 establishes a crucial performance bound. It shows that the risk of the optimally reprogrammed local model, $\mathcal{R}_L(\mathcal{D}^T,\mathbf{P}^*)$, can closely approach that of an optimally reprogrammed service model, $\mathcal{R}_S(\mathcal{D}^T,\mathbf{Q}^*)$, differing by at most $\epsilon$:

$$\mathcal{R}_L(\mathcal{D}^T,\mathbf{P}^*) - \epsilon \le \mathcal{R}_S(\mathcal{D}^T,\mathbf{Q}^*) \le \mathcal{R}_L(\mathcal{D}^T,\mathbf{P}^*)\,. \tag{3}$$

This result offers a key insight: traditional ZOO-based methods expend considerable resources attempting to directly optimize for a low $\mathcal{R}_S(\mathcal{D}^T,\mathbf{Q}^*)$ via unstable, query-heavy processes on $\mathcal{F}_S$. AReS, however, transforms this challenge. By first achieving a faithful priming (small $\epsilon$), the problem shifts to optimizing $\mathcal{R}_L(\mathcal{D}^T,\mathbf{P}^*)$ on the local, glass-box model $\mathcal{F}_L$. This local optimization using efficient First-Order Optimization (FOO) is more stable and drastically reduces API calls and computation, thus providing a more practical path to leveraging the service model's capabilities.

## 5. Experiments

**Datasets and Models.** We evaluate AReS on ten diverse visual recognition datasets widely used in previous works [6, 33], spanning various domains: fine-grained categorization (Flowers102 [32], StanfordCars [25]), texture recognition (DTD [11]), action recognition (UCF101 [40]), food classification (Food101 [4]), traffic sign recognition (GTSRB [18]), satellite imagery (EuroSAT [16]), animal recognition (OxfordPets [36]), scene classification (SUN397 [52]), and digit recognition (SVHN [31]). We compare our approach with SOTA closed-box VR methods: BAR [47] and BlackVIP [33]. For the VLM experiments, we additionally compare against LLM-Opt [30], a recent baseline applicable only to VLMs. For VLMs, we use CLIP ViT-B/16 [39] as the ser-

Table 3. Accuracy and Efficiency comparison using ViT-B/16 (Service) in Full-shot setting.

| Method | Flowers | DTD | UCF | Food | GTSRB | EuroSAT | Pets | Cars | SUN | SVHN | Avg. | #API (M) | Time (h) |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| VR (glass-box) | 69.6 | 52.1 | 49.7 | 38.7 | 60.6 | 96.7 | 77.1 | 6.2 | 34.0 | 83.3 | 56.8 | 43.4 | 39.6 |
| BAR | 14.8 | 24.3 | 29.5 | 12.2 | 14.1 | 43.1 | 24.9 | 1.0 | 10.6 | 29.6 | 20.4 | 1,724.2 | 668.2 |
| BlackVIP (RN50) | 15.2 | 42.3 | 34.0 | 14.5 | 13.9 | 67.3 | 63.6 | 3.7 | 22.6 | 30.8 | 30.8 | 2,586.2 | 686.9 |
| **AReS (RN50)** | **43.5** | **43.4** | **39.4** | **25.9** | **48.7** | **84.3** | **73.9** | **5.8** | **19.9** | **73.7** | **45.9** | **0.2** | **8.7** |
| BlackVIP (ViT-B/32) | 27.4 | 44.1 | 36.6 | 18.5 | 38.4 | 59.6 | 65.4 | 3.9 | 23.7 | 30.3 | 34.8 | 2,586.2 | 757.6 |
| **AReS (ViT-B/32)** | **53.6** | **46.9** | **41.4** | **27.3** | **58.8** | **95.6** | **65.0** | **4.7** | **26.9** | **83.5** | **50.4** | **0.2** | **22.4** |

vice model and ViT-B/16 as the local encoder in a 16-shot learning setting, following the same configuration as BlackVIP [33] due to VLMs' strong generalization capabilities. We also include zero-shot CLIP performance as an important baseline to measure adaptation gains. For VMs, we employ pre-trained ViT-B/16 [12] and ResNet101 (RN101) [15] as service models with ViT-B/32 and ResNet50 (RN50) as local encoders, adopting the Full-shot setting following BAR [47] since VMs are not as generalizable as VLMs. For all settings using VMs as a service, we employ Bayesian-guided Label Mapping (BLM) [6] as $g_{\text{out}}$. In addition, we further evaluate our methods on the MLLM LLaVA [27], the real-world API GPT-4o [20], and the commercial VM API service Clarifai. Additional details can be found in Appendix 8.

**Implementation Details and Evaluation Metrics.** For the priming stage in AReS, we use the Adam optimizer [23] with a learning rate of 0.001. For local encoder VR, we follow the implementation in [6], using a padding-based visual prompt approach and the Adam optimizer with a learning rate of 0.01. For baselines, we use the officially released implementations with their recommended hyperparameter settings. All experiments are conducted over *three* rounds on a single NVIDIA RTX 6000 Ada GPU. We evaluate each method using three key metrics to assess practical deployment value: (1) Average accuracy across three runs on downstream tasks (**%**), (2) Total training and inference API calls required in millions (**#API (M)**), and (3) Total wall-clock training time in hours (**Time (h)**) across all datasets. For experiments on real-world APIs, we also report the total financial cost in US dollars ($) for both training and inference.

## 5.1. Experimental Results

**Results on Vision-Language Model (VLMs).** Following BlackVIP [33], we evaluate our approach using a CLIP ViT-B/16 service model and a ViT-B/16 local encoder, with results presented in Table 2. Our method, **AReS**, achieves a 65.4% average accuracy across ten diverse datasets, outperforming both BAR (61.4%) and BlackVIP (62.9%). We also compare against LLM-Opt [30], a recent method that uses GPT-4 to optimize text prompts; despite its extremely high API costs for both CLIP (5011.3M API calls) and the LLM optimizer (additional ∼$110 USD), it underperforms the zero-shot baseline and is only applicable to VLMs. This superior performance by AReS is achieved with a drastic reduction in computational requirements: only 0.02M API calls for the initial priming, compared to 612.84M for BAR and 754.20M for BlackVIP (a reduction of $> 99.99\%$). Furthermore, inferring all datasets' test sets (e.g., zero-shot) requires 0.12M API calls; our method uses only approximately 1/6 of these calls for priming and then requires no API calls during inference, while still outperforming zero-shot by $+8.2\%$ on average. Training time is also significantly reduced to only 3.7 hours, compared to 185.6 and 197.5 hours for BAR and BlackVIP, respectively (a reduction of $> 98\%$). These efficiency gains are particularly notable on challenging datasets such as SVHN (63.2% vs. 44.4% for BlackVIP and 34.6% for BAR) and EuroSAT (85.7% vs. 73.3% for BlackVIP and 77.3% for BAR). Our performance on Flowers102 is also strong at 86.6%, substantially outperforming BlackVIP (70.6%) and BAR (71.0%), and closely approaching the glass-box VR performance (86.8%) without direct use of the service model.

On certain complex datasets, notably Food101 and Cars, we observe that all evaluated reprogramming methods, including our own (with a failure case analysis provided in Appendix 10.7), struggle to outperform the strong zero-shot CLIP baseline. This suggests a potential inherent limitation of input-level visual reprogramming for these specific domains. This hypothesis is strongly supported by the results in Table 2, where a fully glass-box VR approach is shown to be ineffective, even causing a 4.3% performance *decrease* on Food101 compared to the zero-shot baseline. This insight motivates our **AReS-MS** variant, which functions as an intelligent model selection strategy. By using the priming stage to identify these challenging datasets and defaulting to the zero-shot API, AReS-MS boosts the average accuracy to a new state-of-the-art 69.3% (Table 2), effectively navigating the cost-performance trade-off by applying reprogramming only where it is beneficial.

**Results on Vision Models (VMs).** We further validate our approach using ViT-B/16 as the closed-box service model, paired with ViT-B/32 and RN50 local encoders (details in Table 3; results using an RN101 service model are in Appendix Table 8). Our method consistently outperforms BAR and BlackVIP across all configurations. As shown in the main results, we achieve an average accuracy of 50.4% (with a ViT-B/32 local encoder), compared to significantly lower accuracy for BAR (20.4%) and BlackVIP (34.8%). This superior performance comes with drastically reduced resource

Table 4. Performance and Cost comparison on the open-source MLLM LLaVA, the proprietary API GPT-4o, and the commercial VM API Clarifai, evaluated on the EuroSAT 16-shot benchmark. A zero-shot baseline is not applicable for Clarifai as it is a closed-vocabulary model. Costs are shown in total API calls (#) or US dollars ($). AReS demonstrates significant gains where ZOO-based methods struggle.

| Method | LLaVA | | | | GPT-4o | | | | Clarifai | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Acc. ↑ | Train | Infer. | Total (#) ↓ | Acc. ↑ | Train | Infer. | Total ($) ↓ | Acc. ↑ | Train | Infer. | Total ($) ↓ |
| glass-box | 91.3 | – | – | – | – | – | – | – | – | – | – | – |
| Zero-shot | 40.1 | – | 8100 | 8100 | 59.4 | – | 14.6 | 14.6 | – | – | – | – |
| BAR | 34.1 | $\sim 10^6$ | 8100 | $\sim 10^6$ | 59.1 | 57.6 | 14.6 | 72.2 | 68.1 | 38.4 | 9.7 | 48.1 |
| BlackVIP | 39.4 | $\sim 10^7$ | 8100 | $\sim 10^7$ | 60.1 | 86.4 | 14.6 | 101.0 | 72.1 | 57.6 | 9.7 | 67.3 |
| **AReS** | **73.1** | 160 | 0 | **160** | **87.2** | 0.3 | 0 | **0.3** | **83.2** | 0.2 | 0 | **0.2** |

requirements; in the full-shot setting, our method uses less than 0.02% of the API calls and only 3-7% of the computation time required by competitors like BlackVIP, which demands over 2500M API calls and 750 hours of training.

To further analyze the effectiveness of our priming stage, we compare AReS against a direct glass-box VR baseline performed on the local model. Surprisingly, simply reprogramming the local encoder directly already outperforms the complex ZOO-based BlackVIP (e.g., 45.3% vs. 34.8% for the ViT-B/32 local model in Appendix Table 11). Our priming process then further elevates this strong baseline by a significant margin (e.g., an additional $+5.1\%$ to 50.4%). Moreover, our approach often exhibits synergistic effects, surpassing both the local VR baseline and the estimated glass-box performance of the service model on several datasets. For instance, with an RN101 service and ViT-B/32 local model, on Flowers102 our method achieves 51.3% (vs. 42.3% service glass-box vs. 38.7% local VR). A detailed analysis is in Appendix Sec. 10.5. This demonstrates our method's potential to effectively combine knowledge from both models for enhanced reprogramming outcomes with high efficiency.

**Results on MLLMs and Real-World APIs.** To validate AReS's practical effectiveness and efficiency, we conducted a targeted evaluation on modern services using the EuroSAT 16-shot benchmark. We test AReS against baselines on three distinct models: the open-source MLLM LLaVA, the proprietary API GPT-4o, and the commercial VM API Clarifai. For the commercial APIs, we report the real-world cost in USD, for which glass-box baselines are unavailable. AReS demonstrates substantial gains in both accuracy and efficiency in Table 4, confirming its value in real-world settings.

AReS's strong performance stems from its conceptual strategy shift, which circumvents challenges faced by ZOO-based methods on modern models. On LLaVA, for example, AReS avoids using input perturbations that appear to disrupt the model's delicate vision-language alignment, a problem that causes BAR and BlackVIP's performance to drop below the Zero-shot baseline. Similarly, on the robust GPT-4o, where ZOO-based methods struggle to find effective perturbations, AReS's local reprogramming remains highly effective, achieving a remarkable +27.8% gain over Zero-shot.

Table 5. Ablation study across Service/Local model backbones.

| Method | Service Backbone | | | | |
|---|---|---|---|---|---|
| | RN50 | RN101 | ViT-B/32 | ViT-B/16 | Avg. |
| ZS | 37.5 | 32.6 | 45.2 | 47.9 | 40.8 |
| BAR | 26.9 | 33.5 | 70.3 | 77.2 | 52.0 |
| BlackVIP (RN50) | 51.3 | 50.8 | 62.9 | 68.5 | 58.4 |
| **AReS** (RN50) | **62.2** | **68.5** | **63.2** | **78.6** | **68.1** |
| BlackVIP (ViT-B/16) | 48.4 | 51.3 | 67.9 | 73.3 | 60.2 |
| **AReS** (ViT-B/16) | **81.8** | **81.9** | **82.0** | **85.7** | **82.9** |

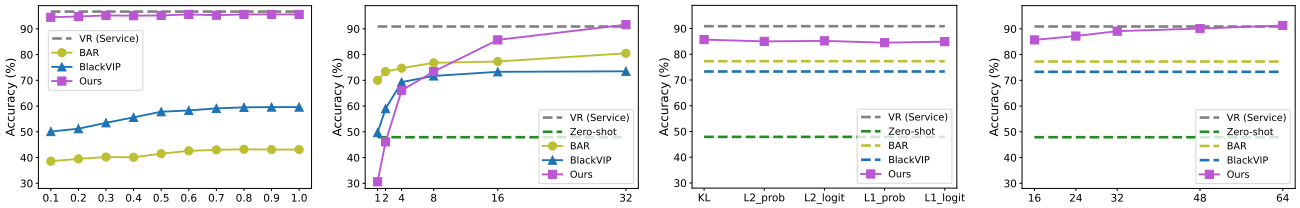
On the commercial Clarifai API, a *closed-vocabulary* model where zero-shot is not an option, AReS again achieves the highest accuracy (83.2%) at a fraction of the cost ($0.2), making it over *300*x *cheaper* than BlackVIP ($67.3). These results confirm that AReS is a robust, effective, and economically viable solution for adapting modern closed-box models. More details in the Appendix Sec. 10.8.

## 5.2. Ablation Study

We present ablation studies in this section. All experiments use EuroSAT with 16 shots per class, CLIP ViT-B/16 (Service) and ViT-B/16 (Local), unless specified otherwise.

**Backbone Versatility.** We examine the effect of varying service model backbones and local encoder choices to demonstrate our approach's versatility when the service model is VLM. The results in Table 5 indicate that our method, whether using a ViT-B/16 or an RN50 local encoder, consistently outperforms baselines like BAR and BlackVIP across diverse CLIP backbones (RN50, RN101, ViT-B/32, and ViT-B/16) used in VLM contexts. For instance, averaged across these service backbones, our method with a ViT-B/16 local encoder achieves an average accuracy of 82.9%, substantially higher than BlackVIP with the same local encoder (60.2%). This underscores the robustness of our method across different architectural choices.

**Component Analysis.** We dissect the contributions of our core components in Table 6: Priming, Visual Reprogramming (VR). We observe that priming the local encoder alone ("Primed") yields 45.6%, which is slightly below the 47.9% zero-shot baseline. It is important to note that this Primed result is not always directly comparable; the priming occurs in the service model's label space (e.g., ImageNet), which




(a) Data Fraction (b) Few-Shot (c) Priming Loss (d) Extra Unlabeled Data

Figure 3. Ablation studies for AReS on the EuroSAT dataset: impact of (a) training data fraction (for VMs), (b) few-shot sample size (for VLMs), (c) priming loss function, and (d) amount of extra unlabeled data for priming on accuracy (%). Default setting involves 16-shot learning, CLIP ViT-B/16 Service, and ViT-B/16 Local encoder unless otherwise indicated by the subplot analysis.

Table 6. Component analysis of Priming, Visual Reprogramming (VR), and Linear Probing (LP).

| Method | Priming | VR | LP | Accuracy |
|---|---|---|---|---|
| Zero-shot | – | – | – | 47.9 |
| Primed | ✓ | – | – | 45.6 |
| Local VR | – | ✓ | – | 70.6 |
| Local LP | – | – | ✓ | 73.8 |
| Local VR+LP | – | ✓ | ✓ | 80.1 |
| **AReS (Ours)** | ✓ | ✓ | – | **85.7** |

can be disjoint from the target task's labels (e.g., EuroSAT). This preparatory focus, which does not require a shared label space, fundamentally distinguishes our approach from conventional knowledge distillation. While the Primed model's accuracy is lower than the zero-shot baseline, an expected outcome, as its performance is bounded by the service model, the true goal is to prepare the local model for reprogramming. Our final AReS result (85.7%) demonstrates that the local model's reprogrammability is significantly enhanced by this step. Local adaptation methods without priming show varied success, with Local VR (70.6%), Local LP (73.8%), and their combination "Local VR+LP" (80.1%) all improving on the baseline. However, our full AReS approach, which combines Priming with Reprogramming, achieves the highest accuracy of 85.7%. This result demonstrates that both components are crucial, outperforming all local-only tuning baselines, and confirms that the priming stage synergistically prepares the local model for a more effective reprogramming.

**Impact of Training Data Size.** The impact of training data size is investigated for both VMs and VLMs. For VMs, Fig. 3a shows our method consistently outperforms BAR and BlackVIP, maintaining significant performance even when data is scarce. More VM evaluation in the 16-shot setting is provided in Appendix Sec. 10.1. For VLMs, varying the number of samples per class in a few-shot setting is presented in Fig. 3b. While at very low shot counts (e.g., $< 8$ shots), the performance of our method can be comparable or slightly underperform some baselines due to suboptimal priming and local VR training from extreme data scarcity, it stabilizes quickly. From 8 shots onwards, our method consistently and significantly outperforms baselines, and with more data (e.g., 32-shots), its performance can be further improved to levels comparable to glass-box service model VR.

**Priming Strategies.** We explore different priming loss functions and the utility of extra unlabeled data, with results in Fig. 3c and Fig. 3d respectively. Among various priming losses (KL divergence, L1/L2 losses on probabilities or logits), KL divergence, used in our main experiments, provides strong and stable performance. While we operate under the strict closed-box assumption (probabilities only), logits-based losses are included for broader comparative analysis. Furthermore, a key benefit of priming is its lack of requirement for extra labeled data, which allows our method to leverage unlabeled downstream task data. As shown in Fig. 3d, increasing the amount of unlabeled data for the priming stage (while keeping VR training fixed at 16-shot) steadily improves our method's performance. This ability to benefit from extra unlabeled data is a unique advantage of our approach compared to BMR baselines (BAR and BlackVIP), which cannot utilize such data in this manner.

## 6. Conclusion

In this work, we addressed key challenges in adapting closed-box Model-as-a-Service APIs, including the heavy query costs, slow optimization, and diminishing effectiveness of ZOO-based methods on modern service models. We introduced AReS, an alternative reprogramming strategy that uses a single-pass interaction with the service API to prepare a local encoder, enabling efficient glass-box reprogramming without further API dependence. This simple, theoretically-supported design yields substantial gains in both performance and practicality. Across diverse benchmarks, AReS consistently outperforms state-of-the-art ZOO approaches on VMs and VLMs, and delivers large improvements on proprietary APIs such as GPT-4o while ZOO-based approaches falter. By dramatically reducing API calls by over 99.99%, AReS enables cost-free inference and offline deployment. These comprehensive findings underscore AReS as a robust, economical, and highly effective solution for closed-box model adaptation. We show that reprogrammability can be efficiently unlocked in a local model through a minimal, one-time priming interaction, facilitating a more practical path for transfer learning in API-centric ecosystems.

## References


[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023. 1, 2

[2] Jimmy Ba and Rich Caruana. Do deep nets really need to be deep? *Advances in neural information processing systems*, 27, 2014. 4

[3] Hyojin Bahng, Ali Jahanian, Swami Sankaranarayanan, and Phillip Isola. Exploring visual prompts for adapting large-scale models. *arXiv preprint arXiv:2203.17274*, 2022. 2, 3, 12

[4] Lukas Bossard, Matthieu Guillaumin, and Luc Van Gool. Food-101–mining discriminative components with random forests. In *ECCV*, 2014. 5, 13

[5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020. 1

[6] Chengyi Cai, Zesheng Ye, Lei Feng, Jianzhong Qi, and Feng Liu. Bayesian-guided label mapping for visual reprogramming. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. 3, 5, 6, 12

[7] Chengyi Cai, Zesheng Ye, Lei Feng, Jianzhong Qi, and Feng Liu. Sample-specific masks for visual reprogramming-based prompting. In *International Conference on Machine Learning (ICML)*, 2024. 1

[8] Aochuan Chen, Yuguang Yao, Pin-Yu Chen, Yihua Zhang, and Sijia Liu. Understanding and improving visual prompting: A label-mapping perspective. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 19133–19143, 2023. 3, 12

[9] Guobin Chen, Wongun Choi, Xiang Yu, Tony Han, and Manmohan Chandraker. Learning efficient object detection models with knowledge distillation. *Advances in neural information processing systems*, 30, 2017. 4

[10] Pin-Yu Chen. Model reprogramming: Resource-efficient cross-domain machine learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 22584–22591, 2024. 1, 2, 12

[11] Mircea Cimpoi, Subhransu Maji, Iasonas Kokkinos, Sammy Mohamed, and Andrea Vedaldi. Describing textures in the wild. In *CVPR*, 2014. 5, 13

[12] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2020. 2, 6

[13] Gamaleldin F Elsayed, Ian Goodfellow, and Jascha Sohl-Dickstein. Adversarial reprogramming of neural networks. In *ICLR*, 2018. 12

[14] Zixian Guo, Yuxiang Wei, Ming Liu, Zhilong Ji, Jinfeng Bai, Yiwen Guo, and Wangmeng Zuo. Black-box tuning of vision-language models with effective gradient approximation. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 5356–5368, Singapore, 2023. Association for Computational Linguistics. 2, 3, 4, 13

[15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016. 2, 6

[16] Patrick Helber, Benjamin Bischke, Andreas Dengel, and Damian Borth. Eurosat: A novel dataset and deep learning benchmark for land use and land cover classification. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 2019. 5, 13

[17] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015. 4

[18] Sebastian Houben, Johannes Stallkamp, Jan Salmen, Marc Schlipsing, and Christian Igel. Detection of traffic signs in real-world images: The german traffic sign detection benchmark. In *IJCNN*, 2013. 5, 13

[19] Daniel Hsu, Ziwei Ji, Matus Telgarsky, and Lan Wang. Generalization bounds via distillation. *arXiv preprint arXiv:2104.05641*, 2021. 4

[20] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*, 2024. 6

[21] Guangda Ji and Zhanxing Zhu. Knowledge distillation in wide neural networks: Risk bound, data efficiency and imperfect teacher. *Advances in Neural Information Processing Systems*, 33:20823–20833, 2020. 4

[22] Menglin Jia, Luming Tang, Bor-Chun Chen, Claire Cardie, Serge Belongie, Bharath Hariharan, and Ser-Nam Lim. Visual prompt tuning. In *European conference on computer vision*, pages 709–727. Springer, 2022. 1, 2, 3, 12

[23] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014. 6

[24] Eliska Kloberdanz, Jin Tian, and Wei Le. An improved (adversarial) reprogramming technique for neural networks. In *ICANN*, 2021. 12

[25] Jonathan Krause, Michael Stark, Jia Deng, and Li Fei-Fei. 3d object representations for fine-grained categorization. In *ICCV workshops*, 2013. 5, 13

[26] Nilakshan Kuna., Jing Zhang, and Mehrtash Harandi. Lavip: Language-grounded visual prompting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 2840–2848, 2024. 4, 13

[27] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *Advances in neural information processing systems*, 36:34892–34916, 2023. 1, 2, 6, 14

[28] Sijia Liu, Bhavya Kailkhura, Pin-Yu Chen, Paishun Ting, Shiyu Chang, and Lisa Amini. Zeroth-order stochastic variance reduction for nonconvex optimization. *Advances in neural information processing systems*, 31, 2018. 1, 3, 13

[29] Sijia Liu, Pin-Yu Chen, Bhavya Kailkhura, Gaoyuan Zhang, Alfred O. Hero III, and Pramod K. Varshney. A primer on zeroth-order optimization in signal processing and machine learning: Principals, recent advances, and applications. *IEEE Signal Processing Magazine*, 37(5):43–54, 2020. 1, 2, 4

[30] Shihong Liu, Samuel Yu, Zhiqiu Lin, Deepak Pathak, and Deva Ramanan. Language models as black-box optimizers for vision-language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12687–12697, 2024. 4, 5, 6, 13, 14

[31] Yuval Netzer, Tao Wang, Adam Coates, Alessandro Bissacco, Baolin Wu, Andrew Y Ng, et al. Reading digits in natural images with unsupervised feature learning. In *NeurIPS workshop*, 2011. 5, 13

[32] Maria-Elena Nilsback and Andrew Zisserman. Automated flower classification over a large number of classes. In *2008 Sixth Indian conference on computer vision, graphics & image processing*. IEEE, 2008. 5, 13

[33] Changdae Oh, Hyeji Hwang, Hee-young Lee, YongTaek Lim, Geunyoung Jung, Jiyoung Jung, Hosik Choi, and Kyungwoo Song. Blackvip: Black-box visual prompting for robust transfer learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 24224–24235, 2023. 1, 2, 3, 4, 5, 6, 12, 13, 14, 15, 20

[34] Yassine Ouali, Adrian Bulat, Brais Martinez, and Georgios Tzimiropoulos. Black box few-shot adaptation for vision-language models. In *Proceedings of the International Conference on Computer Vision (ICCV)*, 2023. 1, 2, 3, 4, 13

[35] Seonghwan Park, Jaehyeon Jeong, Yongjun Kim, Jaeho Lee, and Namhoon Lee. ZIP: An efficient zeroth-order prompt tuning for black-box vision-language models. In *The Thirteenth International Conference on Learning Representations*, 2025. 1, 2

[36] Omkar M Parkhi, Andrea Vedaldi, Andrew Zisserman, and CV Jawahar. Cats and dogs. In *CVPR*, 2012. 5, 13

[37] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilimuntala, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chaudhuri. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, pages 8024–8035, 2019. 15

[38] Mary Phuong and Christoph Lampert. Towards understanding knowledge distillation. In *International conference on machine learning*, pages 5142–5151. PMLR, 2019. 4

[39] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *ICML*, 2021. 1, 2, 5, 15

[40] Khurram Soomro, Amir Roshan Zamir, and Mubarak Shah. Ucf101: A dataset of 101 human actions classes from videos in the wild. *arXiv preprint arXiv:1212.0402*, 2012. 5, 13

[41] James C Spall. Multivariate stochastic approximation using a simultaneous perturbation gradient approximation. *IEEE transactions on automatic control*, 37(3):332–341, 1992. 1, 4, 13

[42] James C Spall. A one-measurement form of simultaneous perturbation stochastic approximation. *Automatica*, 33(1): 109–112, 1997. 1, 4, 13

[43] Samuel Stanton, Pavel Izmailov, Polina Kirichenko, Alexander A Alemi, and Andrew G Wilson. Does knowledge distillation really work? *Advances in neural information processing systems*, 34:6906–6919, 2021. 4

[44] Haotian Sun, Yuchen Zhuang, Wei Wei, Chao Zhang, and Bo Dai. BBox-adapter: Lightweight adapting for black-box large language models. In *Proceedings of the 41st International Conference on Machine Learning*, pages 47280–47304. PMLR, 2024. 1, 2

[45] Tianxiang Sun, Yunfan Shao, Hong Qian, Xuanjing Huang, and Xipeng Qiu. Black-box tuning for language-model-as-a-service. In *Proceedings of ICML*, 2022. 1, 2

[46] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023. 1

[47] Yun-Yun Tsai, Pin-Yu Chen, and Tsung-Yi Ho. Transfer learning without knowing: Reprogramming black-box machine learning models with scarce data and limited resources. In *International Conference on Machine Learning*, pages 9614–9624. PMLR, 2020. 1, 2, 3, 4, 5, 6, 12, 13, 14

[48] Hsi-Ai Tsao, Lei Hsiung, Pin-Yu Chen, Sijia Liu, and Tsung-Yi Ho. AutoVP: An Automated Visual Prompting Framework and Benchmark. In *The Twelfth International Conference on Learning Representations*, 2024. 3, 12

[49] Chun-Chen Tu, Paishun Ting, Pin-Yu Chen, Sijia Liu, Huan Zhang, Jinfeng Yi, Cho-Jui Hsieh, and Shin-Ming Cheng. Autozoom: Autoencoder-based zeroth order optimization method for attacking black-box neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, pages 742–749, 2019. 1, 3, 13

[50] Zhengbo Wang, Jian Liang, Ran He, Zilei Wang, and Tieniu Tan. Connecting the dots: Collaborative fine-tuning for black-box vision-language models. In *Proceedings of International Conference on Machine Learning (ICML)*, 2024. 1, 2, 3, 4, 13

[51] Ross Wightman. Pytorch image models. `https://github.com/rwightman/pytorch-image-models`, 2019. 15

[52] Jianxiong Xiao, James Hays, Krista A Ehinger, Aude Oliva, and Antonio Torralba. Sun database: Large-scale scene recognition from abbey to zoo. In *CVPR*, 2010. 5, 13

[53] Lang Yu, Qin Chen, Jiaju Lin, and Liang He. Black-box prompt tuning for vision-language model as a service. In *IJCAI*, pages 1686–1694, 2023. 1, 2, 3, 4, 13

[54] Runjia Zeng, Cheng Han, Qifan Wang, Chunshu Wu, Tong Geng, Lifu Huang, Ying Nian Wu, and Dongfang Liu. Visual fourier prompt tuning. In *Advances in Neural Information Processing Systems*, pages 5552–5585. Curran Associates, Inc., 2024. 3

[55] Yuexiang Zhai, Shengbang Tong, Xiao Li, Mu Cai, Qing Qu, Yong Jae Lee, and Yi Ma. Investigating the catastrophic forgetting in multimodal large language models. In *NeurIPS 2023 Workshop on Instruction Tuning and Instruction Following*. 14

[56] Guanhua Zhang, Yihua Zhang, Yang Zhang, Wenqi Fan, Qing Li, Sijia Liu, and Shiyu Chang. Fairness reprogramming.

*Advances in neural information processing systems*, 35:34347–34362, 2022. 2, 12

[57] Yihua Zhang, Pingzhi Li, Junyuan Hong, Jiaxiang Li, Yimeng Zhang, Wenqing Zheng, Pin-Yu Chen, Jason D Lee, Wotao Yin, Mingyi Hong, et al. Revisiting zeroth-order optimization for memory-efficient llm fine-tuning: a benchmark. In *Proceedings of the 41st International Conference on Machine Learning*, pages 59173–59190, 2024. 1, 2, 4

[58] Kaiyang Zhou, Jingkang Yang, Chen Change Loy, and Ziwei Liu. Conditional prompt learning for vision-language models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 16816–16825, 2022. 3

[59] Kaiyang Zhou, Jingkang Yang, Chen Change Loy, and Ziwei Liu. Learning to prompt for vision-language models. *International Journal of Computer Vision*, 130(9):2337–2348, 2022. 3

# Alternative Reprogramming for Service Models
# Supplementary Materials

This appendix provides comprehensive details supporting the main paper. Section 7 elaborates on model reprogramming techniques, while Section 8 presents detailed experimental configurations, including dataset statistics and implementation specifics for all baselines and our AReS method (Section 8.3). Section 9 details the complete theoretical analysis establishing performance bounds between service and local models. Additional experimental results are reported in Section 10, covering few-shot VM experiments (Section 10.1), local model enhancement analysis (Section 10.2), evaluations on **real-world proprietary APIs** such as [1]GPT-4o and [2]Clarifai (Section 10.8), and a detailed analysis of challenging VLM scenarios (Section 10.7). Section 11 offers further discussion on our contributions and design rationale. Finally, Section 12 discusses the limitations of our approach.

## 7. Details of Model Reprogramming

Model Reprogramming (MR) [3, 10, 22, 56] is a technique that adapts pre-trained models, such as a source model $\mathcal{F}_S$, to new tasks without altering their underlying parameters. This is particularly useful when dealing with powerful models accessed as a service (MaaS) where direct modification is not possible. The core idea involves an input transformation function $g_{\text{in}}$ that adapts inputs $x^T$ from the downstream task domain $\mathcal{X}^T$ to the source domain $\mathcal{X}^S$ through a learnable prompt $\mathbf{P}$. Additionally, an output or label mapping function $g_{\text{out}}$ is employed to bridge the source label space $\mathcal{Y}^S$ and the target label space $\mathcal{Y}^T$. This approach aims to leverage the rich features learned by large pre-trained models for new, potentially resource-scarce tasks, by essentially "tricking" the model into performing a different function through these carefully crafted input and output manipulations.

### 7.1. Input Transformation

Input transformation [33, 47, 48, 56] in Model Reprogramming, denoted as $g_{\text{in}}(x^T|\mathbf{P})$, serves to bridge the domain gap between the downstream task and the source task the original model $\mathcal{F}_S$ was trained on. It introduces a learnable prompt or program $\mathbf{P}$ that modifies the downstream task inputs $x^T$ before they are fed to $\mathcal{F}_S$, aiming to make the downstream data compatible with its input expectations. Common approaches in Visual Reprogramming (VR) include padding-based VR, which adds trainable noise patterns to the outer frames of an image while preserving its integrity, and watermarking-based VR, where trainable noise patterns are overlaid directly onto the input images. In the context of BAR [47], the input transformation takes the form of an "adversarial program" $P = \tanh(W \odot M)$, where $W$ represents learnable parameters and $M$ is a binary mask ensuring that the original embedded target data remains unchanged. This adversarial program $P$ is universal to all target data samples and is learned to make the model $\mathcal{F}_S$ produce outputs that can be mapped to the desired target task labels. The learning of these parameters, especially in closed-box settings, often relies on zeroth-order optimization techniques.

### 7.2. Output Mapping

Output mapping [6, 8, 24, 47], $g_{\text{out}}$, is a crucial step that translates the predictions from the pre-trained model's original label space $\mathcal{Y}^S$ to the downstream task's label space $\mathcal{Y}^T$, which are often distinct. This process is frequently designed to be gradient-free, meaning it does not introduce additional trainable parameters requiring backpropagation, thus preserving efficiency, especially for tasks with large label spaces. Existing gradient-free methods like Random Label Mapping (RLM) [13], Frequent Label Mapping (FLM) [47], and Iterative Label Mapping (ILM) [8] typically establish a one-to-one correspondence. However, such one-to-one mappings can be limiting as they may overlook more complex, potentially many-to-many relationships between pre-trained and downstream labels. To address this, probabilistic and multi-label mapping strategies have been developed. Bayesian-guided Label Mapping (BLM), for example, constructs a probabilistic matrix quantifying pairwise relationships between pre-trained and downstream labels using Bayesian conditional probability, allowing for a flexible many-to-many mapping. Similarly, BAR [47] can utilize multi-label mapping where multiple source labels map to a single target label, often determined by a frequency-based scheme from initial predictions.

The nature of $g_{\text{out}}$ can differ for VMs and VLMs. For VMs, if label spaces differ, an explicit $g_{\text{out}}$ like BLM is necessary, and in the AReS framework, priming for VMs occurs in the service model's label probability space. For VLMs, $g_{\text{out}}$ might be an identity mapping if label spaces align, or the VLM's text encoder can naturally transform outputs to the downstream task label space.

[1]https://platform.openai.com/docs/models/gpt-4o
[2]https://www.clarifai.com/

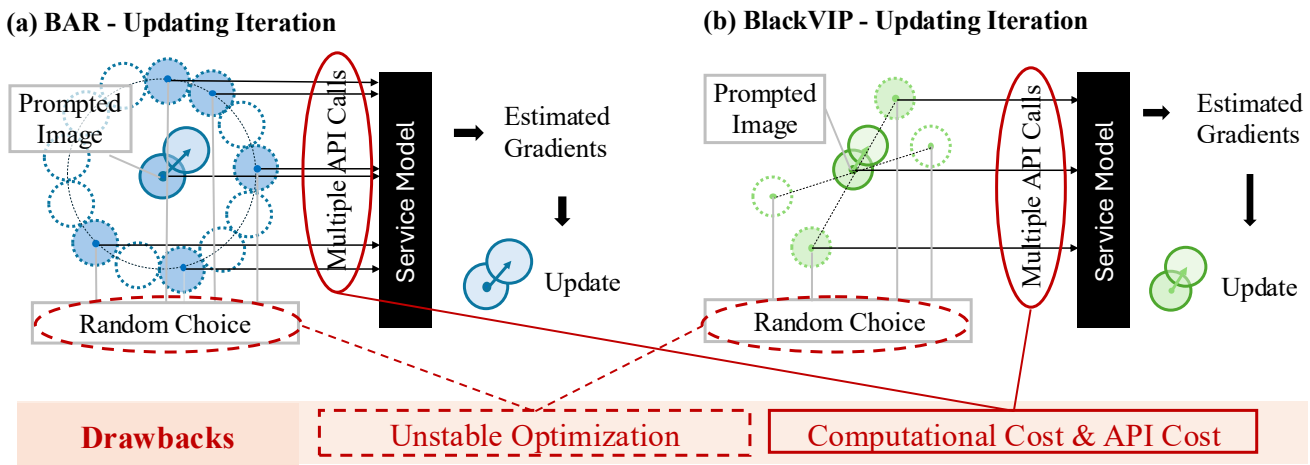



Figure 4. Illustration of Zeroth-Order Optimization (ZOO) techniques commonly used in closed-box model reprogramming. **(a)** BAR with Randomized Gradient-Free (RGF) method estimates gradients by querying the model with random directional perturbations. **(b)** BlackVIP with Simultaneous Perturbation Stochastic Approximation with Gradient Correction (SPSA-GC) approximates gradients using only two model queries with a randomly generated perturbation vector. Both approaches suffer from high query complexity and noisy gradient estimates, leading to unstable and computationally intensive optimization, especially for high-dimensional prompts.

### 7.3. Training

The training process in model reprogramming focuses on learning the parameters of the input transformation $g_{\text{in}}(\cdot|P)$, such as the adversarial program $P$, while the pre-trained source model $\mathcal{F}_S$ remains frozen. The objective is to minimize a loss function on the downstream task data using the transformed inputs and mapped outputs. In a glass-box setting, where gradients from $\mathcal{F}_S$ are accessible, standard gradient-based optimization can efficiently update the transformation parameters. Our AReS method leverages this by first priming a local, fully accessible model $\mathcal{F}_L$, then performing efficient glass-box prompt learning. Conversely, when $\mathcal{F}_S$ is a closed-box, Zeroth-Order Optimization (ZOO) techniques are employed. These methods estimate gradients using only the model's input-output responses, for instance, through random gradient-free (RGF) optimization [28, 49], Simultaneous Perturbation Stochastic Approximation with Gradient Correction (SPSA-GC) [41, 42], as shown in Fig. 4 and Fig. 5, closed-box training presents significant difficulties: optimization can be unstable due to noisy gradient estimates; it incurs high computational costs and numerous API calls; it requires continuous API dependency for both training and inference; and the performance gains are often uncertain despite the substantial investment. Our AReS approach aims to circumvent these challenges by shifting the reprogramming to a locally primed model.

## 8. Experimental Setting

### 8.1. Datasets

**Datasets.** We evaluate our method on ten diverse datasets commonly used in transfer learning literature. These datasets span various domains including fine-grained categorization (Flowers102 [32], StanfordCars [25]), texture recognition (DTD [11]), action recognition (UCF101 [40]), food classification (Food101 [4]), traffic sign recognition (GTSRB [18]), satellite imagery (EuroSAT [16]), animal recognition (OxfordPets [36]), scene classification (SUN397 [52]), and digit recognition (SVHN [31]). Following established protocols in prior work [33], we adopt a few-shot learning setup with 16 randomly selected training examples per class, using the entire test set for evaluation when the service model is VLM. And full-shot learning is applied when the service model is VM, consistent with [47]. Table 7 provides detailed statistics for each dataset.

### 8.2. Experimental Scope and Rationale

Our focus on image classification is a deliberate choice made to ensure a direct and rigorous comparison with the established art in closed-box Model Reprogramming (BMR) [33, 47]. The primary methods in this field, including our main baselines (BAR [47], BlackVIP [33], LLM-Opt [30]) and other related works [14, 26, 34, 50, 53] shown in Table 1, are all benchmarked on classification tasks for both standard Vision Models (VMs) and Vision-Language Models (VLMs). Adhering to this established protocol allows for a fair and robust evaluation of our framework, which is designed to provide an efficient

Table 7. Detailed dataset information.

| Dataset | Full-shot Training | 16-shot Training | Testing Set Size | Number of Classes |
|---|---|---|---|---|
| Flowers102 | 4,093 | 1,632 | 2,463 | 102 |
| DTD | 2,820 | 752 | 1,692 | 47 |
| UCF101 | 7,639 | 1,616 | 3,783 | 101 |
| Food101 | 50,500 | 1,616 | 30,300 | 101 |
| GTSRB | 39,209 | 688 | 12,630 | 43 |
| EuroSAT | 13,500 | 160 | 8,100 | 10 |
| OxfordPets | 2,944 | 592 | 3,669 | 37 |
| StanfordCars | 6,509 | 3,136 | 8,041 | 196 |
| SUN397 | 15,888 | 6,352 | 19,850 | 397 |
| SVHN | 73,257 | 160 | 26,032 | 10 |

reprogramming solution for both model types. Furthermore, classification remains a challenging and relevant testbed; recent work has shown that powerful Multimodal Large Language Models (MLLMs) e.g., LLaVA [27] often suffer from catastrophic forgetting, failing to retain the full classification performance of their underlying visual towers [55]. This highlights the non-trivial nature of adapting these models for pure classification and validates the importance of our approach.

While extending AReS to generative tasks like Visual Question Answering (VQA) is a valuable direction for future research, such tasks are outside the scope of this initial investigation. A core goal of our work is to support standard vision classifiers like ViT and ResNet, for which open-ended generative tasks are not directly applicable. By demonstrating AReS's effectiveness on the fundamental task of classification—a common ground for both VMs and VLMs—we build a strong and reliable foundation for future extensions into more complex, multimodal reasoning tasks.

### 8.3. Baselines and Implementation Details

Our experiments compare AReS against three primary closed-box visual reprogramming baselines: BAR, BlackVIP, and LLM-Opt.

**BAR [47]**[3] repurposes closed-box models by learning a universal adversarial program that is added to target inputs. It relies on Randomized Gradient-Free (RGF) optimization, based on input-output responses, and employs a multi-label mapping scheme. For our VMs experiments, we utilize BLM[4] for BAR to ensure a fair comparison with other methods, including our own, which also uses BLM. We implemented BAR by referencing its official codebase and BlackVIP's reported BAR implementation, adhering to the hyperparameters detailed in BlackVIP's appendix[5] for stable convergence. BAR uses focal loss as its learning objective.

**BlackVIP [33]**[6] enhances closed-box visual prompting by introducing a Coordinator network to generate input-dependent visual prompts and employs Simultaneous Perturbation Stochastic Approximation with Gradient Correction (SPSA-GC) for optimization. We use the official BlackVIP codebase and strictly adhere to the hyperparameter settings reported in their paper and detailed in their configuration files, which cover learning rates and SPSA-GC specific parameters. BlackVIP utilizes cross-entropy loss. For adapting VLMs, all methods, including BlackVIP and BAR, are evaluated in a 16-shot learning setting (16 training samples per class), consistent with the setup in the BlackVIP paper.

**LLM-Opt [30]**[7] utilizes a large language model (e.g., GPT-4) as a closed-box optimizer to find effective text prompts for a target VLM. The method employs an automated "hill-climbing" procedure, where it provides the LLM optimizer with both high- and low-performing prompts as textual feedback to guide the search for better candidates [30]. However, this approach has two significant limitations. First, it introduces a second, costly API dependency for the LLM optimizer, which can amount to over $100 in API fees for a single experimental run [30]. Second, because the method exclusively optimizes the text prompt, its applicability is restricted to VLMs and it cannot be used to adapt standard VMs, which are a key focus of our work.

**AReS**, our proposed method, the visual reprogramming (VR) components are implemented leveraging the publicly available

[3] https://github.com/yunyuntsai/black-box-Adversarial-Reprogramming
[4] https://github.com/tmlr-group/BayesianLM
[5] https://github.com/changdaeoh/BlackVIP/blob/main/docs/configuration.md
[6] https://github.com/changdaeoh/BlackVIP
[7] https://llm-can-optimize-vlm.github.io

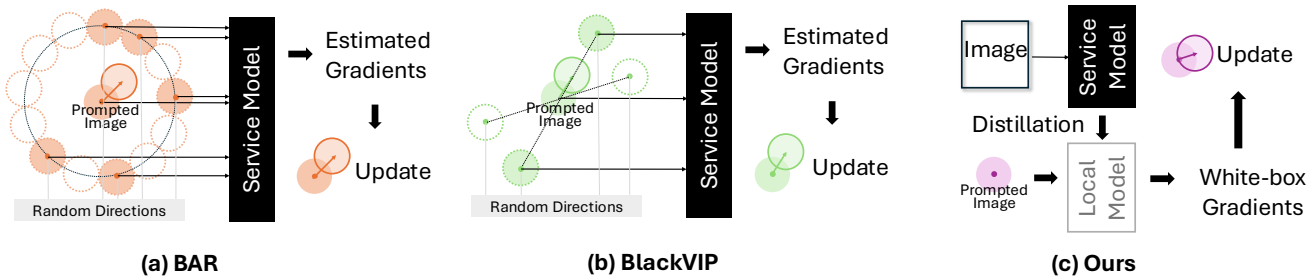



Figure 5. Comparison of closed-box visual reprogramming approaches. **(a)** BAR and **(b)** BlackVIP rely on Zeroth-Order Optimization (ZOO) by repeatedly querying the service model with perturbed inputs (e.g., using random directions) to estimate gradients for updating the visual prompt. These methods suffer from high API call costs and potentially unstable optimization. **(c)** Our AReS method performs a one-time priming from the service model to a local model. Subsequent visual prompt optimization occurs efficiently on this local model using glass-box gradients, eliminating further API calls and enabling stable, cost-effective adaptation.

codebase of BLM. The priming stage, transferring from the service API to the local model's classification head, is performed for 100 epochs. The subsequent glass-box VR on the primed local model is conducted for 200 epochs. Both stages use the Adam optimizer with learning rates of 0.001 for KD and 0.01 for local VR, respectively. The local VR part of AReS employs cross-entropy loss. AReS follows the 16-shot protocol for VLM experiments and operates in a full-shot setting for VM experiments. Following [33], pre-trained Vision Transformer (ViT) models and encoders used in our experiments are sourced from `timm` [51], ResNet models are from `PyTorch` [37], and CLIP models are from the official `CLIP` repository [39].

### 8.4. Clarification of VLM and VM Evaluation Setups

Our experiments distinguish between two primary evaluation setups: one for Vision-Language Models (VLMs) and another for standard Vision Models (VMs). The core differences lie in how each model type handles the output space of the downstream task and the data settings used for evaluation, which were chosen to ensure fair and direct comparisons with established baselines.

**VLM Evaluation Setup.** In our experiments, the VLM setup primarily involves models like CLIP, which is consistent with the standard set by our main baseline, BlackVIP. The key characteristics are:

- **Output Handling:** VLMs utilize a text encoder to perform zero-shot classification without a fixed output head. This means their output space is naturally aligned with the downstream task's text labels. Consequently, a separate or complex label mapping mechanism is not required, simplifying the adaptation process.
- **Data Setting:** To ensure a direct comparison, the evaluation follows BlackVIP's established protocol, which uses a **16-shot** learning setting. The closed-box model reprogramming for all compared methods is learned using only these few-shot labeled samples.

**VM Evaluation Setup.** The VM setup uses standard, pre-trained vision models with a fixed output vocabulary, such as an ImageNet-pretrained ViT-B/16 or ResNet101. This setup presents a more significant challenge:

- **The Challenge of Label Space Mismatch:** Unlike VLMs, these models face a fundamental label space mismatch. Their fixed output vocabulary (e.g., 1000 ImageNet classes) does not align with the labels of the downstream task (e.g., Flowers102). This is a "non-trivial problem" that prior work like BlackVIP explicitly avoided.
- **AReS's Solution:** Our framework directly solves this challenge by incorporating Bayesian-guided Label Mapping (BLM) to bridge the source and target label spaces. In this setup, AReS's priming stage occurs within the service model's source label space, as it has no inherent knowledge of the target class names.
- **Data Setting:** Following the standard for BMR on vision models established by BAR, these experiments are conducted in a **full-shot** setting. For completeness, a few-shot VM setting was also explored in our appendix.

## 9. Details of Theoretical Analysis

In this section, we provide a theoretical analysis to understand the effectiveness of our proposed method, AReS. Our framework involves two primary components affecting the final performance on the downstream task: priming from the service model $\mathcal{F}_S$ to the local encoder $\mathcal{F}_L$, and visual reprogramming (VR) applied to the local model using prompt $\mathbf{P}$. We begin our analysis by

considering the scenario where the model's output logits are directly aligned with the target labels, effectively assuming the label mapping $g_{\text{out}}$ is an identity function due to the text encoder.

**Notations:** Let $\mathcal{D}^T$ represent the downstream task data distribution $p(x, y)$ over inputs $x \in \mathcal{X}^T$ and labels $y \in \mathcal{Y}^T$. Let $\mathcal{F}_S : \mathcal{X}^S \to \mathbb{R}^{K^S}$ be the closed-box service model and $\mathcal{F}_L : \mathcal{X}^S \to \mathcal{Z}$ be the local surrogate model. After priming, $\mathcal{F}_L$ includes the learned linear layer mapping inputs to the downstream task's logit space $\mathcal{Z} = \mathbb{R}^{K^T}$. The visual reprogramming involves an input transformation $g_{\text{in}}(x, p)$ parameterized by $p$. $\ell : \mathcal{Z} \times \mathcal{Y}^T \to \mathbb{R}_{\geq 0}$ is cross-entropy loss.

Let $p_S(x)$ and $p_L(x)$ denote the probability distributions generated by the service model $\mathcal{F}_S$ and the local model $\mathcal{F}_L$ for an input $x$, respectively. For the purpose of this theoretical analysis, we introduce logits. Let $z_S(x)$ and $z_L(x)$ be the logits produced by the service model $\mathcal{F}_S$ and the local model $\mathcal{F}_L$ for input $x$ *before* visual reprogramming. It is important to note that this assumption of direct logit access from $\mathcal{F}_S$ is made for theoretical tractability and differs from the practical setting in the main paper, where only probabilities are assumed to be accessible from the service model. Let $z_S^*(x, \mathbf{Q}^*) = \mathcal{F}_S(g_{\text{in}}(x, \mathbf{Q}^*))$ and $z_L^*(x, \mathbf{P}^*) = \mathcal{F}_L(g_{\text{in}}(x, \mathbf{P}^*))$ be the logits produced by the service model and local model, respectively, for input $x$ *after* visual reprogramming with their respective optimal prompts $\mathbf{Q}^*$ and $\mathbf{P}^*$.

The downstream risk for the local model $\mathcal{F}_L$ *without* VR is:

$$\mathcal{R}_L(\mathcal{D}^T) := \mathbb{E}_{(x,y)\sim\mathcal{D}^T}[\ell(z_L(x), y)]$$

The downstream risk for the local model $\mathcal{F}_L$ *after* applying VR with its optimal prompt $\mathbf{P}^*$ is:

$$\mathcal{R}_L(\mathcal{D}^T, \mathbf{P}^*) := \mathbb{E}_{(x,y)\sim\mathcal{D}^T}[\ell(z_L^*(x, \mathbf{P}^*), y)]$$

Similarly, for the service model $\mathcal{F}_S$, the downstream risk *without* VR is:

$$\mathcal{R}_S(\mathcal{D}^T) := \mathbb{E}_{(x,y)\sim\mathcal{D}^T}[\ell(z_S(x), y)]\,.$$

And the downstream risk for the service model $\mathcal{F}_S$ *after* applying VR with its optimal prompt $\mathbf{Q}^*$ is:

$$\mathcal{R}_S(\mathcal{D}^T, \mathbf{Q}^*) := \mathbb{E}_{(x,y)\sim\mathcal{D}^T}[\ell(z_S^*(x, \mathbf{Q}^*), y)]\,.$$

**Definitions and Assumptions.** We introduce the following definitions and assumptions for our theoretical analysis.

**Definition 1.** ($\epsilon$-Faithful Priming). The priming from $\mathcal{F}_S$ to $\mathcal{F}_L$ is considered $\epsilon$-faithful if the expected $L_1$ norm of the difference between their output logits is bounded by $\epsilon \geq 0$, both before and after applying their respective optimal visual reprogramming prompts:

$$\mathbb{E}_{(x,y)\sim\mathcal{D}^T}[\|z_S(x) - z_L(x)\|_1] \leq \epsilon$$

and

$$\mathbb{E}_{(x,y)\sim\mathcal{D}^T}[\|z_S^*(x, \mathbf{Q}^*) - z_L^*(x, \mathbf{P}^*)\|_1] \leq \epsilon\,.$$

This implies that the priming is sufficiently effective such that the logit distributions of the service and local models are closely aligned. A smaller $\epsilon$ indicates a more faithful priming.

**Assumption 1.** *(Service Model Superiority). The service model $\mathcal{F}_S$ is inherently more powerful or better aligned with the downstream task distribution $\mathcal{D}^T$ compared to the local model $\mathcal{F}_L$, both before and after optimal visual reprogramming. Formally:*

$$\mathcal{R}_S(\mathcal{D}^T) \leq \mathcal{R}_L(\mathcal{D}^T)$$

*and*

$$\mathcal{R}_S(\mathcal{D}^T, \mathbf{Q}^*) \leq \mathcal{R}_L(\mathcal{D}^T, \mathbf{P}^*)\,.$$

*This is a natural assumption that motivates the use of the service model.*

**Assumption 2.** *($\epsilon$-Faithful Priming). The priming process is effective, resulting in the local model $\mathcal{F}_L$ closely mimicking the logit distributions of the service model $\mathcal{F}_S$, both immediately after priming (before VR) and after both models have been optimally reprogrammed for the downstream task.*

### 9.1. Service and Local Model Performance Difference Bound

This section establishes the theoretical foundation for understanding the performance relationship between service and local models in our AReS framework. We begin by proving that the cross-entropy loss function exhibits Lipschitz continuity with respect to logit differences (Lemma 1), which provides the mathematical basis for our subsequent analysis. Building on this property, we derive performance difference bounds (Lemma 2) that quantify how faithful priming affects the performance gap between models. Finally, we establish tight bounds on service model performance relative to the primed local model (Theorem 3), demonstrating that faithful priming enables the local model to closely approximate the service model's capabilities both before and after optimal visual reprogramming.

**Lemma 1.** *(Lipschitz Continuity of Cross-Entropy Loss with respect to Logits). The cross-entropy loss function $\ell(z, y) = -\sum_{j=1}^{K^T} y_j \log(p_j(z))$, where $p_j(z)$ are softmax probabilities derived from logits $z \in \mathbb{R}^{K^T}$ and $y$ is a one-hot true label vector, is Lipschitz continuous with respect to the logits $z$. The specific Lipschitz constant depends on the norm used to measure the difference between logit vectors. Specifically: $\ell(z, y)$ is 1-Lipschitz with respect to the $L_1$ norm of the logits:*

$$|\ell(z_1, y) - \ell(z_2, y)| \leq \|z_1 - z_2\|_1 ,$$

*for any two logit vectors $z_1, z_2$. These constants do not explicitly depend on the number of classes $K^T$ (for $K^T \geq 2$).*

*Proof.* The gradient of the cross-entropy loss $\ell(z, y)$ with respect to the logits $z$ is given by:

$$\nabla_z \ell(z, y) = p(z) - y ,$$

where $p(z)$ is the vector of softmax probabilities derived from $z$, and $y$ is the one-hot true label vector. A differentiable function $f(x)$ is $L$-Lipschitz with respect to a norm $\|\cdot\|_p$ if the dual norm $\|\cdot\|_q$ of its gradient is bounded by $L$ (i.e., $\|\nabla f(x)\|_q \leq L$), by Hölder's inequality. For the $L_1$ norm of logits ($p = 1$, dual norm $q = \infty$): We examine the $L_\infty$ norm of the gradient:

$$\|\nabla_z \ell(z, y)\|_\infty = \|p(z) - y\|_\infty = \max_{j=1,\ldots,K^T} |p_j(z) - y_j| .$$

Let $k$ be the index of the true class ($y_k = 1$, and $y_j = 0$ for $j \neq k$). Then $|p_k(z) - 1| = 1 - p_k(z) \leq 1$. For $j \neq k$, $|p_j(z) - 0| = p_j(z) \leq 1$. Thus, $\|\nabla_z \ell(z, y)\|_\infty \leq 1$. This establishes $L = 1$.

□

**Lemma 2.** *(Performance Difference Bound due to Priming Faithfulness). Under Assumption 2 ($\epsilon$-Faithful Priming), the difference in performance between the local model $\mathcal{F}_L$ and the service model $\mathcal{F}_S$ is bounded by $\epsilon$ as follows:*

1. *Before visual reprogramming:*

$$\mathcal{R}_L(\mathcal{D}^T) - \mathcal{R}_S(\mathcal{D}^T) \leq \epsilon ;$$

2. *After optimal visual reprogramming with prompts $\mathbf{P}^*$ for $\mathcal{F}_L$ and $\mathbf{Q}^*$ for $\mathcal{F}_S$:*

$$\mathcal{R}_L(\mathcal{D}^T, \mathbf{P}^*) - \mathcal{R}_S(\mathcal{D}^T, \mathbf{Q}^*) \leq \epsilon .$$

*Proof.* We will prove the first part of the lemma regarding performance before visual reprogramming. The proof for the second part (after optimal visual reprogramming) follows an identical structure, applying the arguments to $z_L^*(x, \mathbf{P}^*)$ and $z_S^*(x, \mathbf{Q}^*)$ and using the corresponding condition from Assumption 2.

The difference in downstream risks between the local model $\mathcal{F}_L$ and the service model $\mathcal{F}_S$ is:

$$\begin{aligned}\mathcal{R}_L(\mathcal{D}^T) - \mathcal{R}_S(\mathcal{D}^T) &= \mathbb{E}_{(x,y)\sim\mathcal{D}^T}[\ell(z_L(x), y)] - \mathbb{E}_{(x,y)\sim\mathcal{D}^T}[\ell(z_S(x), y)] \\ &= \mathbb{E}_{(x,y)\sim\mathcal{D}^T}[\ell(z_L(x), y) - \ell(z_S(x), y)] .\end{aligned}$$

By Lemma 1, the cross-entropy loss $\ell(z, y)$ is 1-Lipschitz continuous with respect to the $L_1$ norm of its logit inputs. Therefore, for any specific sample $(x, y)$:

$$\ell(z_L(x), y) - \ell(z_S(x), y) \leq |\ell(z_L(x), y) - \ell(z_S(x), y)| \leq \|z_L(x) - z_S(x)\|_1 .$$

Taking the expectation over the data distribution $\mathcal{D}^T$:

$$\mathbb{E}_{(x,y)\sim\mathcal{D}^T}[\ell(z_L(x), y) - \ell(z_S(x), y)] \leq \mathbb{E}_{(x,y)\sim\mathcal{D}^T}[\|z_L(x) - z_S(x)\|_1] .$$

Table 8. Accuracy and Efficiency comparison using RN101 (Service) in Full-shot setting.

| Method | Flowers | DTD | UCF | Food | GTSRB | EuroSAT | Pets | Cars | SUN | SVHN | Avg. | #API (M) | Time (h) |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| VR (glass-box) | 42.3 | 44.4 | 34.4 | 25.4 | 53.1 | 85.0 | 73.9 | 5.0 | 20.5 | 75.1 | 45.9 | 43.4 | 21.9 |
| BAR | 16.9 | 25.3 | 30.2 | 13.9 | 22.5 | 45.7 | 28.7 | 1.5 | 10.7 | 42.1 | 23.8 | 1,724.2 | 313.5 |
| BlackVIP (RN50) | 21.7 | 39.6 | 31.4 | 14.9 | 26.8 | 68.3 | 61.4 | 3.5 | 12.3 | 42.6 | 32.3 | 2,586.2 | 320.5 |
| **AReS (RN50)** | **41.3** | **42.3** | **37.2** | **25.3** | **48.4** | **84.3** | **73.7** | **5.1** | **19.7** | **72.9** | **45.0** | **0.2** | **8.7** |
| BlackVIP (ViT-B/32) | 26.7 | 37.7 | 30.2 | 13.3 | 33.8 | 67.3 | 57.9 | 3.0 | 14.5 | 34.5 | 31.9 | 2,586.2 | 418.0 |
| **AReS (ViT-B/32)** | **51.3** | **45.8** | **41.5** | **30.2** | **57.9** | **94.9** | **63.3** | **5.1** | **25.6** | **83.6** | **49.9** | **0.2** | **22.3** |

From Assumption 2 ($\epsilon$-Faithful Priming), we have the condition that $\mathbb{E}_{(x,y)\sim\mathcal{D}^T}[\|z_S(x) - z_L(x)\|_1] \leq \epsilon$. Substituting this into the inequality:

$$\mathcal{R}_L(\mathcal{D}^T) - \mathcal{R}_S(\mathcal{D}^T) \leq \epsilon\,.$$

□

**Theorem 3.** *(Bounding Service Model Performance). Combining Lemma 2 with Assumption 1 (Service Model Superiority), we can establish bounds on the performance of the service model $\mathcal{F}_S$ relative to the primed local model $\mathcal{F}_L$:*

1. *Before visual reprogramming:*

$$\mathcal{R}_L(\mathcal{D}^T) - \epsilon \leq \mathcal{R}_S(\mathcal{D}^T) \leq \mathcal{R}_L(\mathcal{D}^T)\,; \tag{4}$$

2. *After optimal visual reprogramming:*

$$\mathcal{R}_L(\mathcal{D}^T, \mathbf{P}^*) - \epsilon \leq \mathcal{R}_S(\mathcal{D}^T, \mathbf{Q}^*) \leq \mathcal{R}_L(\mathcal{D}^T, \mathbf{P}^*)\,. \tag{5}$$

*Proof.* By Lemma 2, we have $\mathcal{R}_L(\mathcal{D}^T) - \mathcal{R}_S(\mathcal{D}^T) \leq \epsilon$, which rearranges to $\mathcal{R}_L(\mathcal{D}^T) - \epsilon \leq \mathcal{R}_S(\mathcal{D}^T)$. From Assumption 1, we have $\mathcal{R}_S(\mathcal{D}^T) \leq \mathcal{R}_L(\mathcal{D}^T)$. Combining these two inequalities gives:

$$\mathcal{R}_L(\mathcal{D}^T) - \epsilon \leq \mathcal{R}_S(\mathcal{D}^T) \leq \mathcal{R}_L(\mathcal{D}^T)\,.$$

The reprogrammed case can be shown similarly. □

The above theoretical analysis provides solid justification for AReS's effectiveness by showing that faithful knowledge priming ($\epsilon$-faithful) directly translates to bounded performance differences between service and local models. Theorem 3's key insight reveals that traditional closed-box methods expend significant computational resources attempting to optimize the service model's performance directly, while AReS transforms this challenge by first achieving faithful priming (small $\epsilon$) and then efficiently optimizing the local model using stable first-order methods. This theoretical framework validates our empirical findings and explains why AReS can achieve competitive performance with dramatically reduced API calls and computational costs.

## 10. Additional Experimental Results

### 10.1. VM as a Service Model in Few-Shot Setting

Table 9 details the performance of VM adaptation using ViT-B/16 as the service model and ViT-B/32 as the local model within a challenging 16-shot per class setting. This contrasts with the full-shot experiments for VMs presented in the main paper (Tables 8 and 3), which are generally preferred for VMs due to their more limited generalization compared to VLMs like CLIP. These few-shot results, however, provide a valuable benchmark for performance in data-scarce conditions. As anticipated, all evaluated methods show a decline in performance relative to the full-shot scenario. Notably, the closed-box reprogramming methods BAR and BlackVIP achieve average accuracies of only 14.6% and 25.9%, respectively. Despite this challenging low-data regime, our AReS method demonstrates superior performance, achieving an average accuracy of 32.8%. This represents a significant average improvement of +18.2 percentage points over BAR and +6.9 percentage points over BlackVIP. This consistent outperformance, even with extremely limited training data, underscores AReS's effectiveness in robustly transferring knowledge from closed-box service models and significantly enhancing local model reprogramming capabilities across diverse datasets and varying data availability.

Table 9. Accuracy comparison using ViT-B/16 (Service) and ViT-B/32 (Local) in 16-shot setting.

| Method | Flowers | DTD | UCF | Food | GTSRB | EuroSAT | Pets | Cars | SUN | SVHN | Avg. |
|---|---|---|---|---|---|---|---|---|---|---|---|
| VR (glass-box) | 55.0 | 44.7 | 42.0 | 18.0 | 17.4 | 70.6 | 70.7 | 5.7 | 29.6 | 28.4 | 38.2 |
| BAR | 9.7 | 16.3 | 23.5 | 8.1 | 4.8 | 34.2 | 21.5 | 1.0 | 7.6 | 19.6 | 14.6 |
| BlackVIP | 16.1 | **36.1** | 33.8 | 12.5 | 9.1 | 54.9 | 54.5 | 3.9 | 22.1 | 16.2 | 25.9 |
| **AReS (Ours)** | **46.0** | 35.5 | **34.3** | **12.7** | **19.6** | **71.1** | **54.3** | **4.2** | **23.0** | **27.3** | **32.8** |

Table 10. Accuracy comparison between Local VR (ViT-B/16) and our method on VLMs using CLIP ViT-B/16 (Service) and ViT-B/16 (Local) in a 16-shot setting.

| Method | Flowers | DTD | UCF | Food | GTSRB | EuroSAT | Pets | Cars | SUN | SVHN | Avg. |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Zero-shot | 71.3 | 43.9 | 66.9 | **85.9** | 21.0 | 47.9 | **89.1** | **65.2** | 62.6 | 17.9 | 57.2 |
| Local VR | 55.0 | 44.7 | 42.0 | 18.0 | 17.4 | 70.6 | 70.7 | 5.7 | 29.6 | 28.4 | 38.2 |
| **AReS (Ours)** | **86.6** | **48.2** | **67.1** | 68.8 | **39.4** | **85.7** | 88.9 | 43.2 | **62.8** | **63.2** | **65.4** |

Table 11. Accuracy comparison between Local VR and our method on VMs in Full-shot setting.

| **Service** | **Local** | **Local VR** | **Ours** |
|---|---|---|---|
| ViT-B/16 | ViT-B/32 | 45.3 | **50.4** |
| | RN50 | 43.9 | **45.9** |
| RN101 | ViT-B/32 | 45.3 | **49.9** |
| | RN50 | 43.9 | **45.0** |

Table 12. Comparison of trainable parameters and accuracy on the EuroSAT dataset. More parameters do not correlate with better performance for ZOO-based methods.

| **Method** | **# Trainable Params** | **Accuracy (%)** |
|---|---|---|
| VP w/ SPSA-GC | 69K | 70.9 |
| BAR | 37K | 77.3 |
| BlackVIP | 9K | 73.3 |
| **AReS (Ours)** | **21K** | **85.7** |

## 10.2. AReS's Impact on Local Model Performance

To quantify AReS's enhancement of local model capabilities, we compare it against directly performing glass-box VR on the local model (termed "Local VR"). For AReS, we assume a pre-trained local encoder, consistent with methods like BlackVIP. For this evaluation of the local encoder only, we additionally assume a pre-trained linear layer on the source domain. As shown in Table 10 for VLMs (CLIP ViT-B/16 service, ViT-B/16 local, 16-shot), AReS achieves a 65.4% average accuracy, a substantial $+27.2\%$ improvement over Local VR's 38.2%. This highlights that AReS's priming significantly boosts the local ViT-B/16's reprogrammability beyond its standalone capacity.

A similar advantage for AReS is observed with VMs in the full-shot setting, as detailed in Table 11. For instance, with a ViT-B/16 service model, AReS improves upon Local VR by $+5.1\%$ (50.4% vs. 45.3%) when using a ViT-B/32 local model, and by $+2.0\%$ (45.9% vs. 43.9%) with an RN50 local model. Consistent gains are also seen with an RN101 service model. These results across both VLM and VM configurations underscore the critical role of AReS's initial priming phase. By effectively transferring knowledge from the more powerful service model, AReS significantly elevates the local model's baseline performance and its potential for reprogramming, demonstrating a more effective utilization of combined model strengths.

## 10.3. Dissecting the Source of Performance Gain

The superior performance of AReS stems not from a higher parameter count or the mere inclusion of local tuning, but from its novel two-stage framework. We demonstrate that the initial priming stage is an indispensable component and that the overall design is highly parameter-efficient.

Table 13. Comparison with an enhanced BlackVIP baseline on the EuroSAT dataset. The results confirm that AReS's performance gain is primarily from its superior two-stage framework.

| Method | Accuracy (%) |
|---|---|
| Local VR | 70.6 |
| BlackVIP | 73.3 |
| BlackVIP w/ Local Tuning | 74.1 |
| **AReS (Ours)** | **85.7** |

Table 14. Accuracy (%) comparison on the EuroSAT dataset across different few-shot settings. AReS consistently outperforms the strong local baseline (Local VR+LP) as data availability increases.

| Method | 4-shot | 8-shot | 16-shot | 32-shot |
|---|---|---|---|---|
| BlackVIP | 69.3 | 71.7 | 73.3 | 72.9 |
| Local VR+LP | 59.7 | 68.1 | 80.1 | 86.5 |
| **AReS (Ours)** | **66.1** | **73.4** | **85.7** | **91.6** |

First, an analysis of parameter efficiency reveals that for ZOO-based methods, a larger number of trainable parameters does not guarantee better performance and can even be detrimental. As shown in Table 12, a baseline using SPSA-GC with a large visual prompt (69K parameters) achieves a lower accuracy than more compact models. This is likely because noisy gradient estimates are less stable in higher-dimensional spaces. In contrast, AReS's stable, glass-box optimization allows for the effective tuning of a compact prompt, confirming its effectiveness is due to a superior optimization strategy, not parameter quantity.

Second, the performance gain is critically dependent on the initial priming stage. Our component analysis on EuroSAT (Table 6 in the main paper) shows that local visual reprogramming alone (Local VR) achieves 70.6% accuracy. The full AReS method, which includes priming, elevates this to 85.7%, a substantial +15.1% improvement demonstrating that the knowledge transferred during priming is essential for unlocking the local model's full potential.To further validate this, we created an enhanced version of BlackVIP that incorporates a local pre-training stage. To mimic a local pre-training stage, we trained its prompt-generating Coordinator decoder with a reconstruction loss, as direct supervision is not possible without a ground-truth prompt. As shown in Table 13, this local optimization provides only a marginal improvement to BlackVIP (+0.8%). AReS still significantly outperforms this enhanced baseline by +11.6%, confirming that the performance gain originates from our superior two-stage framework, where priming makes the local model significantly more amenable to reprogramming.

### 10.4. Performance Scaling with Data Availability

A key question is whether AReS's advantage over a strong, locally-trained baseline persists as more training data becomes available. To investigate this, we conducted an ablation study on the EuroSAT dataset, varying the number of training samples from 4 to 32 shots. The results in Table 14 show that AReS consistently outperforms both the ZOO-based baseline (BlackVIP) and a strong local baseline (Local VR+LP) across all data settings.

While the local baseline's performance improves steadily with more data, it never surpasses AReS. Our method is not only competitive in low-data regimes but also scales more effectively as data increases. For instance, at 32 shots, AReS achieves 91.6% accuracy, maintaining a significant +5.1% lead over the 86.5% from the local baseline. This confirms that the initial priming stage provides a durable advantage that local training alone cannot replicate, equipping the local model with a superior foundation that enables a higher performance ceiling.

### 10.5. Synergistic Effect of Combining Model Knowledge

In certain configurations, AReS's performance can surpass that of a glass-box reprogramming approach on the service model itself. This outcome stems from a synergistic effect, where our framework effectively combines the distinct knowledge of the powerful service model with the unique inductive biases of the local model. Unlike methods [33] that use the local model as a simple feature generator, AReS's two-stage design first primes and then leverages the local model's capabilities during reprogramming. This process allows the final model to correctly classify samples that neither the service model nor the standalone local model could handle individually.

This synergistic effect is demonstrated quantitatively in our analysis of the Flowers102 dataset (Table 15), where we compare scenarios with service models of varying strength. When the service model (RN101) has capabilities comparable to the local model, AReS's final accuracy surpasses both, largely by learning to classify an additional 4.8% of samples that were

Table 15. Illustrative Breakdown of AReS's Performance Composition on Flowers102. The local model for both scenarios is ViT-B/32 (glass-box Acc: 38.7%). "Kept correct" refers to samples the local model classified correctly that AReS also classifies correctly. "Newly learned" refers to samples the local model misclassified that AReS now classifies correctly.

| **Initial State** | **Service: RN101 (42.3%)** | | **Service: ViT-B/16 (69.6%)** | |
|---|---|---|---|---|
| | **% of Dataset** | **How AReS Performed** | **% of Dataset** | **How AReS Performed** |
| Both Service & Local Correct | 30.0% | 29.5% kept correct | 35.0% | 34.0% kept correct |
| Only Local Correct | 8.7% | 7.0% kept correct | 3.7% | 0.5% kept correct |
| Only Service Correct | 12.3% | 10.0% newly learned | 34.6% | 19.0% newly learned |
| Both Incorrect | 49.0% | 4.8% newly learned | 26.7% | 0.1% newly learned |
| **AReS Final Accuracy** | **51.3%** | | **53.6%** | |

Table 16. Accuracy (%) comparison on ImageNet in a 16-shot setting with CLIP ViT-B/16 as the service model. AReS significantly outperforms all baselines.

| **Method** | **ImageNet Acc (%)** | **Gain over Zero-shot (%)** |
|---|---|---|
| VR (glass-box) | 67.4 | +0.7 |
| Zero-shot | 66.7 | - |
| BAR | 64.6 | -2.1 |
| VP w/ SPSA-GC | 62.3 | -4.4 |
| BlackVIP | 67.1 | +0.4 |
| **AReS (Ours)** | **80.1** | **+13.4** |

incorrect for both models initially. Conversely, when the service model (ViT-B/16) is significantly stronger, AReS's role shifts to highly effective knowledge transfer, yielding a remarkable +14.9% absolute improvement over reprogramming the local model alone. This analysis confirms that AReS facilitates a potent combination of knowledge, creating synergistic outcomes not possible with other approaches.

### 10.6. Performance on Large-Scale Benchmarks: ImageNet

To further validate our method's effectiveness on a large-scale, stable benchmark, we conducted experiments on ImageNet in a 16-shot setting, using CLIP ViT-B/16 as the service model. The results, presented in Table 16, demonstrate AReS's significant superiority. While prior ZOO-based methods like BAR and VP w/ SPSA-GC degrade performance compared to the zero-shot baseline, and the SOTA BlackVIP provides only a marginal +0.4% gain, AReS achieves a remarkable 80.1% accuracy. This represents a substantial +13.4% improvement over the zero-shot baseline and a +13.0% gain over the next best competitor, BlackVIP. Notably, AReS even surpasses the glass-box reprogramming baseline, further highlighting the powerful synergistic effect of combining the service model's knowledge with the local model's inductive biases. This is particularly notable as it demonstrates that when a strong local model is available, AReS can effectively employ this advantage to achieve performance superior to both the service model and other local-model-based methods. In contrast, the BlackVIP baseline, despite utilizing the same local encoder architecture, fails to leverage its capabilities, providing only a negligible improvement. These compelling results on a challenging, large-scale dataset confirm that our conceptual shift away from the ZOO-based paradigm is a more robust and effective strategy for closed-box model adaptation.

### 10.7. Performance Analysis on Challenging VLM Scenarios

While AReS demonstrates considerable average performance improvements with unparalleled efficiency, its efficacy with VLMs like CLIP as a service can exhibit variability on particularly challenging datasets, notably Food101 and Cars, as indicated in Table 2. The inherent difficulty of these datasets for reprogramming is underscored by the performance of even highly capable baselines. For instance, glass-box VR performed directly on the CLIP ViT-B/16 service model achieves 81.6% on Food101, which is below the zero-shot performance of 85.9%. On Cars, glass-box VR obtains 66.2%, only a marginal improvement over the 65.2% zero-shot accuracy. This suggests a ceiling for reprogramming efficacy on these complex domains, even with full model access. Consequently, ZOO-based closed-box methods like BAR and BlackVIP also struggle; on Food101, BAR (84.4%) and BlackVIP (85.9%) offer negligible to no improvement over zero-shot performance. Similarly, on Cars, BAR (63.0%) underperforms the zero-shot baseline, and BlackVIP (65.4%) provides only a minimal gain. These

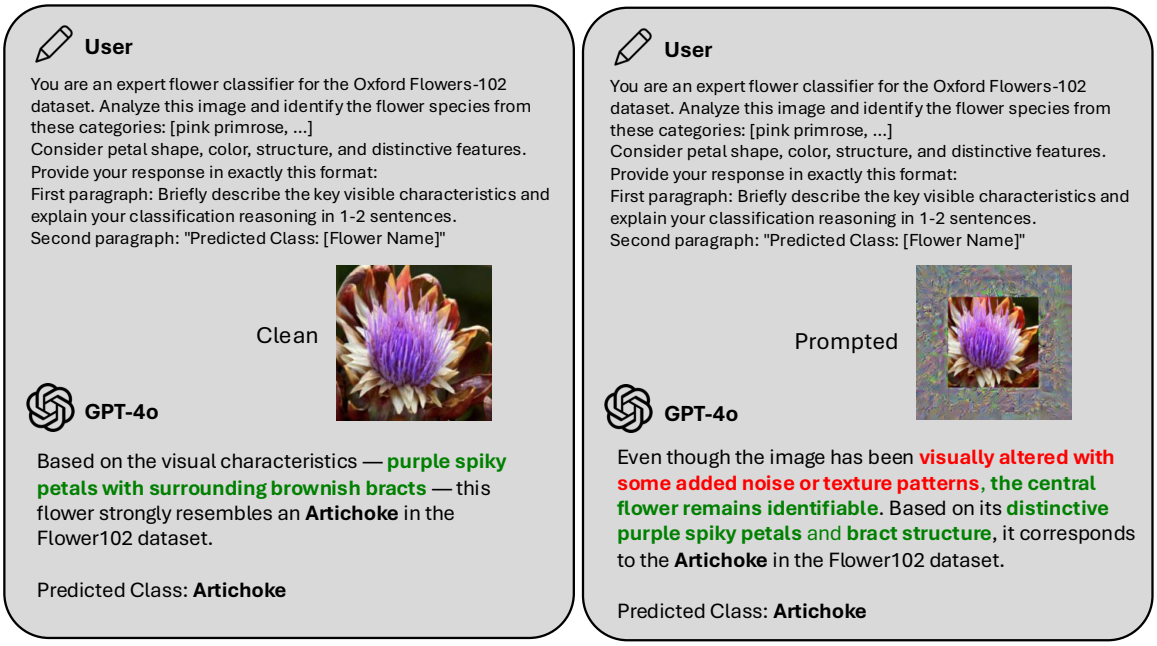



Figure 6. An example of GPT-4o's robustness to the input perturbations used in reprogramming. When presented with a clean image (left) and a prompted image (right), GPT-4o's textual reasoning explicitly acknowledges the visual alteration but correctly identifies the flower's key features in both cases. This demonstrates that the model is often invariant to the input noise that ZOO-based methods rely on, motivating a strategic shift in adaptation methods.

results highlight that substantial API call volumes and computational efforts by existing closed-box methods do not reliably translate into significant performance gains on such challenging datasets.

In this context, AReS's performance (68.8% on Food101 and 43.2% on Cars in Table 2) is logically influenced by the capabilities of the local ViT-B/16 model. Given that our approach entirely avoids API calls during inference and subsequent reprogramming, its success is intrinsically tied to how well the primed knowledge empowers the local model for the specific downstream task. On these particularly demanding datasets, the primed local model may not fully capture the intricate features that the larger service model leverages for its strong zero-shot performance, leading to a performance gap. However, it is crucial to evaluate AReS's contribution in terms of enhancing the local model's standalone reprogrammability. As shown in Table 10, direct Local VR on the ViT-B/16 achieves only 18.0% on Food101 and a mere 5.7% on Cars. AReS elevates these figures to 68.8% and 43.2% respectively, marking substantial improvements of $+50.8\%$ and $+37.5\%$. This significant enhancement of the local model's utility, achieved with minimal, one-time API interaction and drastically reduced computation, is of immense practical value, particularly in scenarios where continuous and costly reliance on service APIs is untenable. Addressing the remaining performance gap on these specific challenging datasets, possibly through advancements in primed techniques or by employing more capable local architectures, presents an interesting direction for future research.

### 10.8. Results on MLLMs and Real-world APIs

To validate AReS's practical effectiveness on modern models, we conducted a targeted evaluation using the EuroSAT 16-shot benchmark on three distinct services: the open-source MLLM LLaVA, the proprietary VLM GPT-4o, and the commercial VM API [8]Clarifai. Since MLLMs do not produce output probabilities by default, we instructed the models to act as a classifier and return a confidence score for each class (e.g., `"For the classes [...], provide a confidence score from 0 to 1 for each."`). Our evaluation reveals a critical limitation of traditional BMR on these services. As illustrated in Figure 6, powerful models like GPT-4o are highly robust to the input perturbations central to ZOO-based reprogramming. The model's textual reasoning shows it can "see past" the visual noise to the underlying image content, rendering the ZOO process that relies on these perturbations ineffective. This finding, supported by quantitative results where ZOO fails to improve over the zero-shot baseline (Table 4), requires a strategic shift away from perturbation-based adaptation

[8]https://www.clarifai.com/

for modern APIs.

In contrast, AReS's two-stage framework is architecturally immune to this issue. By performing a one-time priming step and shifting all subsequent reprogramming to a local model, AReS's success is not dependent on the service model's sensitivity to input noise. This allows it to achieve a remarkable **+27.8%** gain over the zero-shot baseline on GPT-4o, a task where other methods stagnate. Furthermore, on the commercial Clarifai API—a closed-vocabulary setting where zero-shot is not an option—AReS again achieves the highest accuracy (83.2%) at a fraction of the cost ($0.20), making it over **300x cheaper** than BlackVIP ($67.30). These results confirm that AReS's strategy provides a more robust, effective, and economically viable solution for adapting modern closed-box models.

## 11. Further Discussion on Contributions and Design Rationale

We further clarifies the core contributions of AReS, its design rationale, and its positioning relative to alternative paradigms.

### 11.1. On Novelty: Priming for Reprogrammability vs. Traditional Distillation

A key contribution of AReS is the high-level 'prime-then-reprogram' conceptual framework. This framework is flexible, and the priming stage is not constrained to a single implementation. While this work successfully uses an objective function inspired by knowledge distillation (KL divergence), the core idea is to enhance the local model's reprogrammability. Any future technique that can effectively prime a local model's amenability to reprogramming from a closed-box source could be integrated into this framework. This focus on "**priming for reprogrammability**" is what fundamentally distinguishes AReS from traditional distillation (also in Sec. 4):

1. Traditional KD aims to create a final, high-performance student model. This approach often fails in BMR scenarios, particularly when adapting standard Vision Models (VMs). For instance, if the service model is a VM (e.g., pre-trained on ImageNet) and the target is Flowers102, their **label spaces are disjoint**. One cannot directly distill a Flowers102 classifier from an ImageNet classifier.
2. AReS's Priming is **solely a preparatory step**. It is not intended to solve the final task. Instead, it operates in the service model's label space (e.g., ImageNet) to make the local model API-aware when given downstream data (e.g., Flowers102 images).

This novel preparatory focus is what enables the second, local reprogramming stage to be highly effective, even in challenging disjoint-label VM scenarios where *traditional distillation is not applicable*.

### 11.2. On Practicality: Cost-Free Inference as a Core Design Goal

A core design choice of AReS is to perform a single-pass knowledge transfer, which then **intentionally eliminates all subsequent API dependency** during inference. This is not a shortcoming but the central advantage of our framework, enabling practical deployment in on-device, edge, offline, or cost-sensitive scenarios. The practical motivation for this new paradigm is threefold:

1. **ZOO-based Methods are Less Effective for Modern APIs:** We find that the ZOO-based paradigm is becoming ineffective on modern, robust APIs. Powerful models like GPT-4o are often less sensitive to the input perturbations that ZOO methods rely on . Our experiments (Table 4) confirm this: ZOO-based methods provide little to no improvement over the zero-shot baseline on GPT-4o. In contrast, AReS achieves a **+27.8% gain**, succeeding precisely where the previous paradigm fails. This provides a strong practical reason to shift to a local model primed by the service API.
2. **Synergistic Performance:** The AReS framework can unlock synergistic effects by combining the knowledge of the powerful service model with the inductive biases of the local model. As shown in our analysis (Table 15), this allows AReS to correctly classify samples that neither the service model nor the standalone local model could handle individually. This synergy is so effective that AReS can even **outperform the glass-box VR performance of the service model**—the theoretical performance ceiling for any ZOO-based method.
3. **Enabling Real-World Deployment:** This design unlocks a wide range of practical scenarios where perpetual API access is not feasible or desirable, such as on-device or edge-computing applications, scenarios requiring real-time or offline adaptation, and cost-sensitive applications where a >99.99% reduction in API calls is a primary requirement.

Our real-world API experiments (Table 4) confirm this total practical value. On the commercial Clarifai API, AReS achieves superior accuracy (83.2%) for just $0.20, while BlackVIP costs $67.30 for lower accuracy (72.1%).

### 11.3. On Efficacy: AReS vs. Standalone Local Model Reprogramming

To isolate the contribution of our priming stage, we conducted extensive component analyses comparing AReS to a Local VR baseline. This baseline represents the naive approach of simply reprogramming the local model without any priming.

The results conclusively demonstrate that our performance gain is not merely from "just training a small network," but from the synergistic "**prime-then-reprogram**" framework.

1. On EuroSAT (Table 6): Local VR (baseline) achieves 70.6% accuracy. The full AReS framework achieves 85.7%—a +15.1% gain directly attributable to the priming stage.
2. On VLMs (Table 9): AReS achieves a 65.4% average, a +27.2% improvement over the 38.2% from Local VR.
3. On VMs (Table 11): AReS (50.4%) improves upon Local VR (45.3%) by +5.1%.

This empirically proves that our priming stage successfully transfers knowledge from the service model, making the local model significantly more amenable to effective reprogramming.

### 11.4. On the Theoretical Framework

Our theoretical analysis provides the formal justification for why our two-stage approach is effective. The assumption of $\epsilon$-Faithful Priming (Assumption 2) is not a given; it is the explicit goal of our Priming stage. Our framework is constructive:

1. Principle: We posit that if $\epsilon$-faithful priming can be achieved, the unstable, query-heavy closed-box optimization on the service model $\mathcal{F}_S$ can be provably bounded by an efficient, stable, first-order optimization on the local model $\mathcal{F}_L$ (Theorem 3).
2. Mechanism: The Primeing stage is the practical mechanism we designed to achieve this faithful priming (i.e., a small $\epsilon$). We also empirically validate the robustness of this mechanism in our ablation study (Fig. 3c). The results show that our practical, probability-only (closed-box) method achieves the same strong performance as a variant using logits (translucent-box), confirming that our theoretical analysis is well-grounded and its assumptions do not create a gap with our practical implementation.
3. Result: Our extensive empirical results—especially the +27.8% gain on GPT-4o where ZOO methods fail —demonstrate that this mechanism succeeds in practice.

The theory, therefore, explains why our practical approach of shifting optimization to a local model is a sound and effective strategy for leveraging the service model's capabilities. To the best of our knowledge, this is **the first work to provide a formal theoretical analysis for model reprogramming** in the context of closed-box service models.

## 12. Limitations

Our AReS method, while offering benefits in efficiency and local model enhancement, has certain limitations. In extreme data scarcity scenarios, such as 1/2-shot learning, the one-time priming phase may be constrained (see Fig. 3b). This can challenge the $\epsilon$-faithful primed assumption in our theoretical analysis, potentially affecting the optimality of subsequent local VR and leading to suboptimal performance. Furthermore, AReS's performance is inherently affected by the representational capacity of the chosen local model. If the local model, even after priming, cannot capture the complexities of a particularly challenging downstream task, AReS may not achieve the performance levels of methods that continuously leverage the full computational power and feature richness of the larger service model throughout adaptation (discussed in Sec. 10.7).